



Arm CoreSight Protocol Decoder

BO26EB-43

Georg Ansgar Skeide - Candidate 325

Johan Romarheim - Candidate 174

Saleh Barakat - Candidate 190

Advisor - Endre Håland

May 20, 2026



Document Control

Report title: Project report for BO26EB-43 - "Arm CoreSight Protocol Decoder"	Date/Version May 20, 2026 / Ver. 1.0
	Report number: ELN_06
Authors: Georg Ansgar Skeide Johan Romarheim Saleh Barakat	Field of study: Electronics
	Number of pages incl. appendices: 111
University supervisor: Endre Håland	Classification: Open
Remarks: We permit this report to be published.	

Client: HVL	Client reference:
Contact persons: Volker Stolz Eivind Vågslid Skjæveland	Contact information: volker.stolz@hvl.no eivind.vagslid.skjeveland@hvl.no

Abstract

Modern embedded systems face a growing threat from control flow attacks such as Return-Oriented Programming (ROP) and buffer overflows, which subvert legitimate program execution to gain unauthorized control. Detecting such attacks in real time, without imposing overhead on the monitored system, is an open challenge in embedded security.

CoreSight provides a hardware infrastructure for non-intrusive processor tracing that captures every control flow-changing instruction with zero impact on application performance. By decoding this trace stream on-chip in real time, it becomes possible to enforce Control Flow Integrity (CFI) — flagging execution paths that deviate from a pre-determined Control Flow Graph (CFG) — and halt the system the moment an attack is detected.

This work presents the design and implementation of the first stage of a hardware-based CFI pipeline targeting the Xilinx Zynq-7000 System on a Chip (SoC): a Trace Port Interface Unit (TPIU) frame deformatter implemented in VHSIC Hardware Description Language (VHDL) on the Programmable Logic (PL) fabric. The Cortex-A9 Central Processing Unit (CPU) outputs trace data through its TPIU in 16-byte formatted frames; the deformatter strips this framing and recovers a stream of source-tagged trace bytes, ready for downstream decoding and analysis. Operating within a strict four-clock-cycle budget to match the TPIU throughput, the design processes trace data entirely in hardware without involving the host CPU.

Correctness is validated against the open source CoreSight Decoding Library (openCSD), across approximately 25 KB of real multi-source trace data, with byte-for-byte agreement. This work lays the foundation for a complete real-time CFI enforcement system capable of detecting anomalous control flow and triggering an immediate system halt via the CoreSight Cross Trigger Interface (CTI).

Acknowledgements

We would like to thank Associate Lecturer Eivind Vågslid Skjæveland and Professor Volker Stolz of the Department of Computer Science, Electrical Engineering and Mathematical Sciences (Institutt for datateknologi, elektroteknologi og realfag) at Western Norway University of Applied Sciences (Høgskulen på Vestlandet), Bergen, for their academic guidance and support.

We would also like to express our gratitude to Senior Engineer Endre Håland, our supervisor, for his guidance and support throughout the ARM CoreSight project.

Contents

Document Control	1
Acronyms	8
1 Introduction	10
1.1 Scope and contributions	11
1.2 Problem statement	12
1.3 The structure of this report	13
2 ARM CoreSight Architecture	14
2.1 System Overview	14
2.2 Debug Network	15
2.3 Cross Trigger Network	16
2.4 Trace Network	16
2.4.1 Trace Sources	16
2.4.2 Trace Links	18
2.4.3 Trace Sinks	18
2.5 CoreSight Frame Format	20
3 System Design and Architecture	21
3.1 System Overview	21
3.2 Tools	22
3.2.1 Vivado	22
3.2.2 Vitis	23
3.2.3 Integrated Logic Analyzer	23
3.2.4 Yocto	23
3.2.5 CoreSight access library (CSAL)	23
3.2.6 OpenCSD	24
3.2.7 Zynq-7000 SoC	24
3.2.8 Tool Command Language (TCL)	25
3.2.9 Risk Analysis	25
3.2.10 Design Requirements	26

4	Design and VHDL-logic	27
4.1	Structure	27
4.2	Frame generator	28
4.3	Frame Deformatter v1.0	29
4.4	Converting to axi	32
5	Simulation and Verification	34
5.1	Verification Strategy	34
5.2	Individual VHDL testbench	34
5.2.1	Frame Generator Test-1:	34
5.2.2	Frame Generator Test-2:	35
5.2.3	Frame Generator Test-3:	35
5.2.4	Frame Deformatter test with standard ARM CoreSight Frame	36
5.3	Integration: BRAM Test Infrastructure	37
5.3.1	Processor System Reset Hold Time	38
5.3.2	BRAM output latency	38
5.3.3	Testing the Whole Design via HDL Wrapper	39
5.3.4	Full Pipeline Verification Against OpenCSD	39
5.4	Simulation and Verification Summary	40
6	Hardware Integration and Validation	42
6.1	Configuring the TPIU in Vivado	42
6.2	Software Environment: Linux or Baremetal	43
6.2.1	Linux with Yocto	43
6.2.2	Baremetal with Vitis	44
6.3	ROM table Component addresses	44
6.4	Configuring the CoreSight Components	45
6.5	Hardware Validation Results	46
6.5.1	TPIU Pre-Enable Behaviour	46
6.5.2	Observed Stream Characteristics	46
6.5.3	End-to-End Comparison	47
6.6	Implementation reports	47
6.6.1	Timing Report	47
6.6.2	Utilization report	48
7	Discussion	50
7.1	Achievement of design goals	50
7.2	Design decisions and their trade-offs	51
7.3	Future Work	51
8	Conclusion	53

A	Github repository	57
B	Block diagrams	58
B.1	BRAM to Frame Generator	61
B.2	bram address counter	62
B.3	Frame Generator	62
B.4	Frame Deformatter	64
B.5	AXI Master	76
B.6	VITIS code	78
B.7	C# Comparison Script	85
B.8	TCL script for simulation	87
C	ILA simulation	90
D	Risk Analysis	92
E	Design Requirements Analysis	95
E.1	Functional requirements	95
E.2	Non-Functional requirements	96
E.3	Progress plan	97
F	Use of AI Tools	98

List of Figures

1.1	Control Flow Graph Example	11
1.2	The big picture of the joint cybersecurity project, divided into 3 stages.	12
2.1	Typical CoreSight Debug & Trace Architecture for Heterogeneous SoCs	15
2.2	Illustration of the CoreSight 16-byte frame format.	20
3.1	Total system for our project	22
3.2	AMD Zynq-7000 SoC architecture [1].	25
4.1	CoreSight trace capture pipeline on the Zynq-7000 SoC	27
4.2	Logical right shift.	28
4.3	State diagram for the deformatter	29
4.4	Logic split across 4 clock cycles	30
4.5	F-bit concatenation for case statement F.	30
4.6	Example case "01" – byte 2 is Data, byte 0 is ID	31
4.7	Delayed-ID carry example across IDLE – Cycle_A	31
4.8	Deformatter output channels	32
4.9	Advanced eXtensible Interface (AXI) Protocol Finite State Machine (FSM) and connections of the AXI network	33
5.1	Frame Generator testbench output showing incorrect word ordering	34
5.2	Wrong: appends to Least Significant Bit (LSB), reversing frame order	35
5.3	Correct: shifts to Most Significant Bit (MSB), preserving frame order	35
5.4	Frame Generator testbench output showing correct word ordering	35
5.5	Frame Generator testbench output showing HALFWORD IGNORED	35
5.6	Frame Generator testbench output showing RESET ACTIVE	36
5.7	Example formatter frame [2]	36
5.8	Frame deformatter testbench output using the Technical Reference Manual (TRM) example frame	37
5.9	Simplified BRAM Dataflow Block Diagram	37
5.10	PS reset signal fluctuating on startup [3].	38
5.11	Observed 1-cycle delay producing 32 bits of zeroes on first output	38
5.12	Full pipeline simulation using the .coe file.	39

5.13	Excerpt of the channel output CSV recorded by the TCL script.	40
6.1	CSLS script crawls through the Read Only Memory (ROM) table to discover device addresses.	45
6.2	Vitis application block diagram	46
B.1	BRAM infrastructure block diagram	59
B.2	TPIU block diagram	60
C.1	Full ila simulation	91
E.1	Progress plan	97

List of Tables

4.1	The four possible combinations of the v_selector vector.	30
6.1	Summary of the key trade-offs between linux and baremetal.	44
6.2	Vivado Timing Summary at 125 MHz	47
6.3	Longest timing path	48
6.4	Vivado Resource Utilization Summary on Zedboard	48
6.5	Comparison of decoders, with only the worst-case scenario for byte/cycle. B stands for buffer based. Adapted from [4].	49
D.1	Risk Analysis	94
E.1	Functional requirements for the system	95
E.2	Non-Functional requirements for the system	96

Acronyms

AMBA Advanced Microcontroller Bus Architecture. 17

APB Advanced Peripheral Bus. 17

API Application Programming Interface. 23, 24

ATB AMBA Trace Bus. 15, 16, 17, 18, 19, 21, 45

AXI Advanced eXtensible Interface. 5, 11, 19, 21, 22, 23, 24, 27, 32, 33, 43, 44, 47, 53

BRAM Block RAM. 34, 38, 39, 40, 47, 53

CFG Control Flow Graph. 1, 10

CFI Control Flow Integrity. 1, 10, 49

CPU Central Processing Unit. 1, 23, 24

CSAL CoreSight Access Library. 15, 21, 23, 43, 44, 45, 46, 47, 53

CTI Cross Trigger Interface. 1, 11, 15, 16, 17

CTM Cross Trigger Matrix. 16

DAP Debug Access Port. 15

DDR Double Data Rate. 22, 24, 32

DMA Direct Memory Access. 11, 21, 22, 24, 27, 32, 33, 43, 44, 46, 47, 53

ECT Embedded Cross Trigger. 16

EMIO Extended Multiplexed I/O. 18, 19, 21, 24, 42

ETB Embedded Trace Buffer. 11, 14, 18, 19, 21, 32, 43, 44, 45, 47

ETF Embedded Trace FIFO. 19

ETM Embedded Trace Macrocell. 10, 11, 14, 18, 22, 40, 41, 51, 52, 53

ETR Embedded Trace Router. 19, 37

FIFO First in first out. 19, 48

FPGA Field-Programmable Gate Array. 10, 11, 12, 16, 18, 22, 23, 24, 37, 39, 48, 53

FSM Finite State Machine. 5, 23, 29, 30, 33, 41, 47, 53

FTM Fabric Trigger Macrocell. 16

GPIO General Purpose Input/Output. 33, 47

HVL Western Norway University of Applied Sciences. 11

ILA Integrated Logic Analyzer. 23, 46, 47, 53

IP Intellectual Property. 22, 23, 32, 37, 38, 41, 42, 44, 47

JTAG Joint Test Action Group. 23, 24, 44

LSB Least Significant Bit. 5, 20, 35

MIO Multiplexed I/O. 18, 42

MSB Most Significant Bit. 5, 35

openCSD open source CoreSight Decoding Library. 1, 24, 38, 53

PL Programmable Logic. 1, 16, 17, 21, 22, 24, 27, 32, 35, 38, 42

PMU Performance Monitoring Unit. 16

PS Processing Subsystem. 16, 21, 22, 23, 24, 27, 32, 33, 38, 42, 47

PTM Processor Trace Macrocell. 10, 11, 14, 16, 17, 18, 21, 22, 40, 45, 46

ROM Read Only Memory. 6, 14, 15, 44, 45

ROP Return-Oriented Programming. 1, 10

SoC	System on a Chip. 1, 5, 12, 14, 16, 21, 23, 24, 25	TRM	Technical Reference Manual. 5, 34, 36, 37, 40, 46
SRAM	Static Random Access Memory. 19		
		US+	Zynq UltraScale+ MPSoC. 17, 37
TMC	Trace Memory Controller. 14, 19		
TPIU	Trace Port Interface Unit. 1, 3, 11, 12, 18, 19, 21, 22, 23, 24, 26, 27, 28, 32, 33, 34, 37, 39, 41, 42, 43, 45, 46, 47, 48, 49, 50, 51, 52, 53, 95	VHDL	VHSIC Hardware Description Language. 1, 11, 13, 18, 22, 33, 34, 37, 40, 42, 44, 47, 53
		XSA	Xilinx Support Archive. 23

Chapter 1

Introduction

Modern embedded systems are increasingly deployed in security-critical environments where the integrity of software execution cannot be taken for granted [5]. Attacks such as buffer overflows exploit pre-existing software flaws to subvert execution and gain control over software behavior, for example by redirecting a call to a sensitive system function the application was never intended to use [6]. While numerous mitigation techniques have been proposed, they are typically of limited scope, and attackers have demonstrated practical methods to circumvent each of them [7, 8, 9]. Additionally, some mitigations rely on hardware modifications or impose performance overhead, making them impractical for resource-constrained embedded systems [6, 5].

ARM's CoreSight architecture addresses this by providing dedicated on-chip hardware components that passively trace processor execution with zero overhead to the main application [10, 11]. The trace stream produced by CoreSight components such as the Processor Trace Macrocell (PTM) and Embedded Trace Macrocell (ETM) captures every control flow-changing instruction executed by the processor. By decoding this stream in real time on an Field-Programmable Gate Array (FPGA), it becomes possible to detect anomalous control flow the moment it occurs and halt the system before further damage takes place.

This practice is called Control Flow Integrity (CFI), a security policy that dictates software execution must follow a path defined by a pre-determined Control Flow Graph (CFG) [6]. A CFG maps all legitimate control flow transfers in a program.

For example, which functions may call which other functions, and any transfer that falls outside these predefined edges is flagged as a potential attack. Figure 1.1 illustrates this concept, where `MainF` may legitimately call `Fcall`, which in turn may call either `A1` or `B2`. A code reuse attack such as Return-Oriented Programming (ROP) would attempt to redirect execution to an illegitimate target, violating the CFG. By continuously monitoring the trace stream from the ETM and comparing observed control flow against a pre-computed CFG, it becomes possible to detect such violations in real time without imposing any overhead on the host processor [12].

```

1 MainF(int a[], int b[], int
  ↳ len)
2 {
3     Fcall(a, len, A1);
4     Fcall(a, len, B2);
5 }

```

```

1 bool A1(int x, int y) {
2     return x<y;
3 }
4 bool B2(int x, int y) {
5     return x>y;
6 }

```

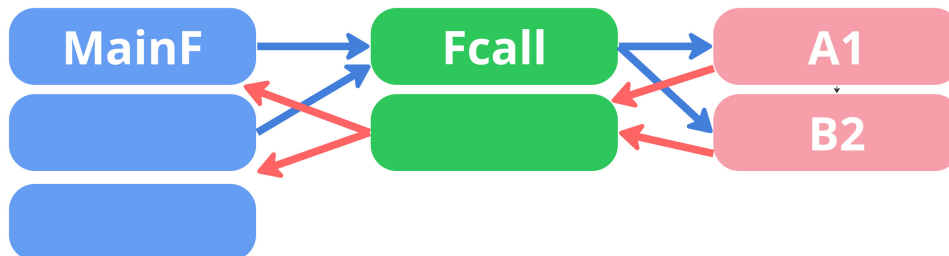


Figure 1.1: Control Flow Graph Example

1.1 Scope and contributions

This project is part of a joint cybersecurity research effort between the Electrical and Software Engineering groups at Western Norway University of Applied Sciences (HVL) and the SUSHI team at CentraleSupélec in Rennes, France [13]. The goal of the collaboration is to develop a complete hardware-based trace decoder pipeline capable of monitoring control flow execution on processors in real time. To this end, we propose a full system concept consisting of three stages: a frame deformatter, an ETM instruction trace decoder, and a trace analyzer that flags abnormal control flow and signals the Cross Trigger Interface (CTI) to shut down the processor, as illustrated in Figure 1.2.

This report covers the design and implementation of the **first stage** of this pipeline: the Trace Port Interface Unit (TPIU) frame deformatter. While the second stage of the broader pipeline concerns an ETM decoder, the Digilent ZedBoard used in this project houses a Zynq-7000 SoC whose Cortex-A9 cores expose PTMs rather than ETMs. The Cortex-A9 processor on the Zynq 7000 platform outputs trace data through its TPIU, which encapsulates trace packets from multiple sources into 16-byte frames. Before any meaningful analysis can be performed, these frames must be deformatted to recover the ETM or PTM trace packets. The deformatter is implemented in VHSIC Hardware Description Language (VHDL) and deployed on the programmable logic of the Zynq 7000, exploiting the tight integration between the processor and the FPGA fabric.

In addition to the deformatter itself, this project also develops a validation platform that enables the correctness of the hardware decoder to be verified against a software reference implementation. Raw trace data captured by the Embedded Trace Buffer (ETB) is decoded on a host PC using OpenCSD, and the result is compared against the output of the hardware deformatter transferred via Advanced eXtensible Interface (AXI) Direct Memory Access (DMA). This platform also lays the groundwork for integration with the subsequent decoder stages.

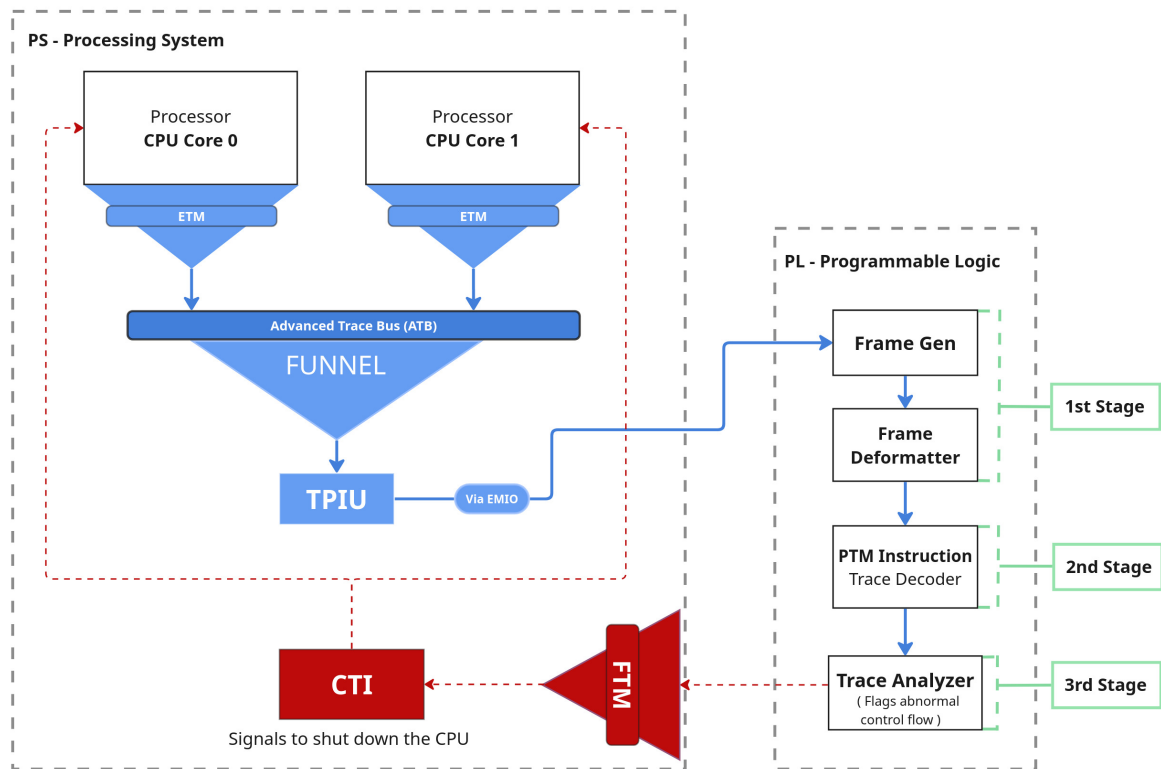


Figure 1.2: The big picture of the joint cybersecurity project, divided into 3 stages.

1.2 Problem statement

Modern System on a Chip (SoC) designs such as the Xilinx Zynq 7000 platform integrate powerful processors with programmable logic. To monitor software integrity in real time without impacting system performance, it is necessary to leverage on-chip hardware components such as ARM CoreSight. Although these components generate continuous execution trace data, the information is encapsulated in a compact and structured format that must be decoded before it can be analyzed.

The TPIU outputs 32 bits of formatted trace data every clock cycle and cannot be paused or throttled. Trace data from multiple sources is interleaved into 16-byte frames, meaning four clock cycles are required to receive a complete frame. Because a new frame will have been assembled every four cycles at maximum throughput, the decoder must also complete its work within those same four cycles to keep pace with the incoming stream. A decoder implemented in software on the host processor would introduce latency and overhead incompatible with real-time security monitoring, making it necessary to implement the decoder directly in the FPGA fabric.

The core research question of this project is therefore:

- "How can the **first stage** of a hardware-based decoder cascade for the ARM CoreSight TPIU frame format be implemented on an FPGA, such that interleaved trace frames are deformatted into a stream of source-tagged data bytes, and how can the resulting data be retrieved and verified against a software reference deformatter?"

1.3 The structure of this report

This report is organized to guide the reader from background theory through to hardware validation. Chapter 1 introduces the project context, motivation, and problem statement. Chapter 2 presents the ARM CoreSight architecture and the relevant components. Chapter 3 describes the overall system design. Chapter 4 details the VHDL implementation. Chapter 5 covers simulation and verification, and Chapter 6 describes hardware integration and validation on the Zedboard. Chapters 7 and 8 discuss the results and present conclusions.

Chapter 2

ARM CoreSight Architecture

2.1 System Overview

ARM CoreSight is a system-wide debug and trace architecture integrated into ARM-based SoCs, providing hardware infrastructure for non-intrusive monitoring of processor execution without impacting system performance [10, 2]. While CoreSight was designed primarily as a debug and trace tool for software developers, the continuous hardware-level visibility it provides into program execution has made it attractive for additional purposes, such as performance profiling, compiler optimization feedback, and runtime security monitoring [6, 4, 14].

As shown in Figure 2.1, the CoreSight architecture consists of three distinct networks. The trace network is the primary functional backbone, responsible for capturing and routing trace data from trace sources to a storage or export sink. The debug network serves as the configuration backbone, providing access to component registers and a Read Only Memory (ROM) table that enumerates all components present in the system.

The cross-trigger network enables coordinated communication between components, for example allowing both processor cores to be halted simultaneously in response to a detected event. The following sections describe each network and their relevant components in the context of this project.

The specific components present in a CoreSight implementation are determined at integration time and vary between devices. For example, older designs may include a PTM and a standalone ETB, while newer designs may instead use an ETM and a Trace Memory Controller (TMC).

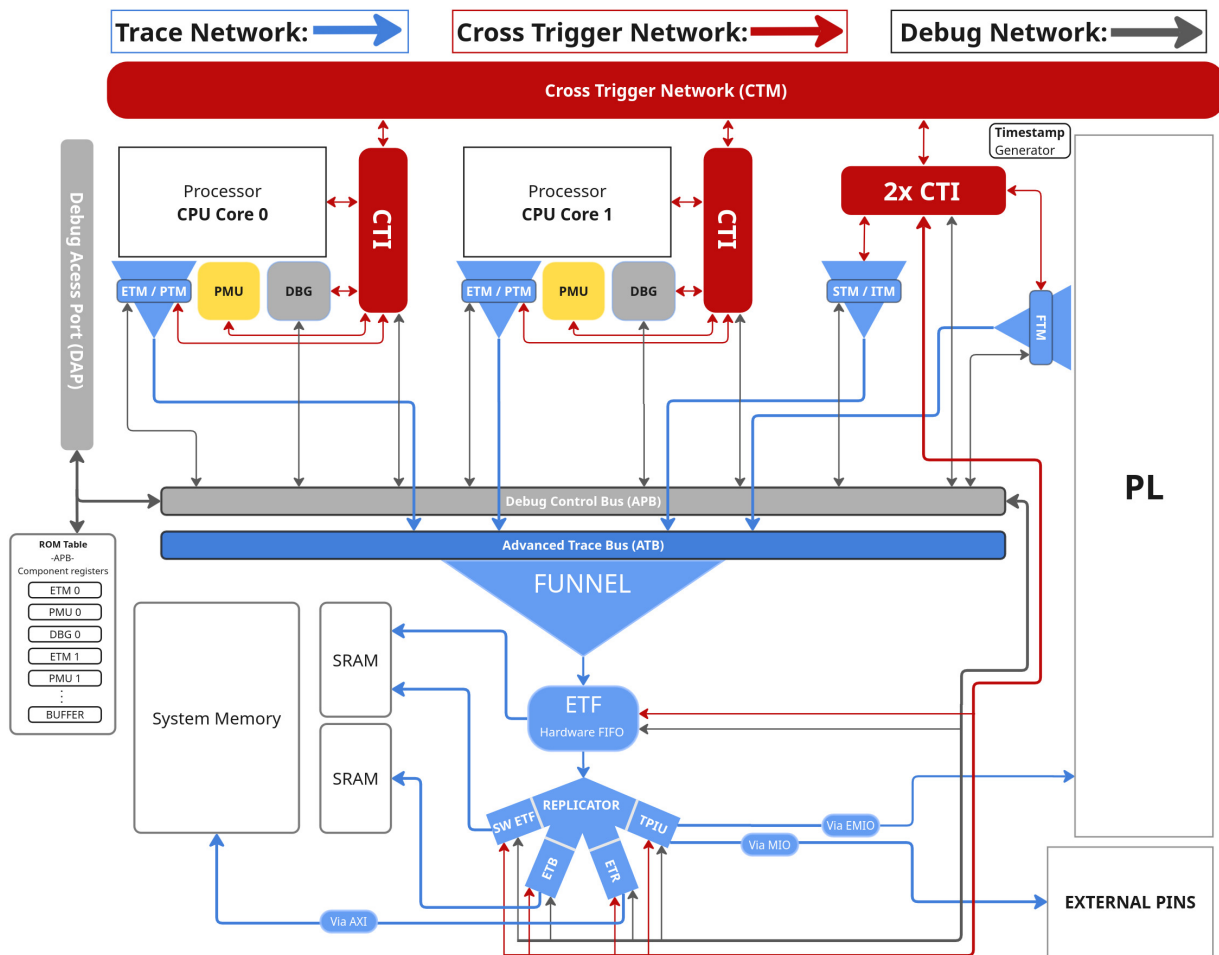


Figure 2.1: Typical CoreSight Debug & Trace Architecture for Heterogeneous SoCs

2.2 Debug Network

The debug network provides external tools and software with the ability to read and write CoreSight component registers, enabling configuration and control of the entire CoreSight system. Access to the debug network is provided through the Debug Access Port (DAP), which bridges an external debugger or software running on the host processor to the internal component address space.

Central to the debug network is the ROM table, a read-only structure residing at a known base address in the system memory map. The ROM table holds a list of offsets pointing to the base addresses of every CoreSight component present in the system. It serves as the entry point for any debug or tracing session — by reading the ROM table, a tool can discover the complete set of available components without requiring prior knowledge of the system topology.

In this project, the CoreSight Access Library (CSAL) is used to interact with the debug network. CSAL provides the `cs1s` utility, which traverses the ROM table and reports the base address and type of each discovered component. This was used to verify the component addresses on the Zynq 7000 platform prior to configuring the trace session, as described in Section 6.4. CSAL further includes both an AMBA Trace Bus (ATB) topology detection algorithm and a CTI detection algorithm, revealing which

components share a direct connection and the port numbers for each connection interface [15].

2.3 Cross Trigger Network

The cross-trigger network is implemented through the Embedded Cross Trigger (ECT) subsystem, which consists of two components: the Cross Trigger Matrix (CTM) and the CTI. One or more CTMs form a system-wide event broadcasting network organised into multiple channels. CTI units are distributed throughout the system, each connected to one or more CoreSight components such as processor cores or Performance Monitoring Unit (PMU)s.

A CTI listens on one or more channels for incoming events, maps them to trigger outputs, and forwards them to its connected components. In the other direction, a CTI also collects triggers from its connected components and broadcasts them as events onto the channel network, enabling bidirectional event propagation across the entire SoC [16].

A practical example relevant to this project is halting both processor cores simultaneously. Each Cortex-A9 core on the Zynq 7000 has an affiliated CTI. By connecting these CTIs through the CTM, a halt event on one core can be broadcast to the other, ensuring both cores stop at the same point. This is particularly important for security monitoring applications, where a detected control flow violation must result in an immediate and complete system halt.

On SoCs such as the Zynq 7000 that include programmable logic, the Fabric Trigger Macrocell (FTM) serves as the interface between the Programmable Logic (PL) and the cross-trigger network. The FTM converts PL signals into ECT trigger signals, allowing logic implemented in the FPGA fabric to inject events into the cross-trigger network. In the context of this project, this means that once the trace analyzer in stage 3 of the pipeline detects an anomalous control flow pattern, it can assert a PL signal through the FTM to the CTI, triggering a system-wide halt of the processor cores without any software intervention.

2.4 Trace Network

2.4.1 Trace Sources

Fabric Trace Macrocell

The FTM discussed in the previous section is not only a trigger interface between the PL and Processing Subsystem (PS), but also a fully fledged trace source. The FTM accepts 32-bit trace data and a 4-bit trace ID directly from the PL fabric via dedicated input pins, formats the incoming data into CoreSight-compatible trace packets, and forwards them onto the ATB to be combined with trace from other sources such as the PTM [16]. This means that arbitrary data produced by custom FPGA logic can be injected directly into the trace stream and routed to the same sinks as processor execution trace, enabling simultaneous tracing of both PS and PL events in a unified stream.

Instrumentation Trace Macrocell (ITM)

The Instrumentation Trace Macrocell (ITM) is a software-driven trace source, in contrast to the PTM which passively monitors processor execution. Software writes values to memory-mapped stimulus registers via the Advanced Peripheral Bus (APB) interface, which are then packaged into Software Instrumentation Trace (SWIT) packets and forwarded onto the ATB.

The primary use cases are printf-style debugging, tracing of OS and application events, and emitting diagnostic system information, with coarse-grained timestamping available to correlate captured events in time. The ITM has largely been superseded by the System Trace Macrocell (STM), which offers greater scalability in terms of supported masters and stimulus ports for more complex multi-core systems. [17]

The STM provides features that the ITM does not support, that is the tracing of hardware events. The STM has 64 hardware event input ports that can be emitted onto the STM trace stream. In the case of the Zynq UltraScale+ MPSoC (US+), the hardware event inputs of the STM are shared by CTI inputs and the remaining 60 hardware event inputs come from the PL. This allows an entire system to be traced, a running application on the processor and simultaneously tracing the state of a block design on the PL [4].

Program Trace Macrocell (PTM)

The Program Trace Macrocell (PTM) is the trace source used in this project. It provides non-intrusive observation of processor execution. The PTM monitors the processor's instruction flow in real time and outputs trace data without interfering with processor execution [18].

The PTM operates according to the Program Flow Trace (PFT) architecture and emits trace packets onto the Advanced Microcontroller Bus Architecture (AMBA) 3 ATB at waypoints, which are points in program execution where control flow changes, such as branches, exceptions, instruction set state changes, and context ID changes. Rather than recording every executed instruction, generating output only at these waypoints significantly reduces trace bandwidth compared to full instruction tracing [18].

To reconstruct the full execution path from these packets, a decoder requires access to the program binary alongside the trace stream. This is because the PTM does not transmit information that can be directly inferred from the binary: only the target addresses of indirect branches and the condition codes of direct branches are provided. All sequential instructions between waypoints are inferred by the decoder from the binary itself.

This design minimises trace bandwidth but introduces a dependency on the binary being available to the decoder. A previous implementation addressed the resulting information gap by storing auxiliary metadata in system memory, which was then read by the hardware at runtime. While this approach incurred negligible performance overhead, it resulted in substantial storage overhead, with experiments showing the required metadata storage could be as large as double the original application

binary itself [12].

The ETM is the successor to the PTM and serves largely the same purpose, though some ETM implementations extend the instruction tracing functionality with additional data tracing support, depending on the target processor. Data tracing records the actual values read from or written to memory during program execution, but is rarely used in practice as it generates significantly larger trace volumes than instruction tracing alone. The Cortex-A9 cores on the Zynq-7000 expose PTMs, which is why this project targets the PTM trace format rather than ETM; however, migrating to an ETM-capable device is the intended direction when developing stages 2 and 3 of the pipeline. PTM offers a stronger trace compression comparing to ETMv3 [4, 19].

2.4.2 Trace Links

Within the CoreSight architecture, trace links are components that sit between trace sources and trace sinks, routing trace data across the ATB infrastructure. The two relevant components are the funnel and the replicator.

Funnel

The trace funnel accepts multiple trace source inputs and merges them into a single output trace stream. This is necessary in multi-source systems where several trace sources, such as multiple PTMs, must drain into a single trace sink. Each input port can be independently enabled or disabled, allowing the system to select which sources contribute to the combined stream. Each trace source is assigned a unique trace ID, which is preserved in the merged stream, allowing a decoder to attribute each packet to its originating trace source.

Replicator

The replicator performs the inverse function: it accepts a single trace input and broadcasts it to multiple outputs. This allows the same trace stream to be routed to more than one sink. For example, to both anETB for on-chip storage and a TPIU for either off-chip export through Extended Multiplexed I/O (EMIO) or routed to the FPGA fabric through EMIO.

2.4.3 Trace Sinks

TPIU

The TPIU is a trace sink which can be routed either to external pins through Multiplexed I/O (MIO), or directly to the FPGA fabric through EMIO. In our project this is the most crucial component, as it allows trace data to directly enter as input to VHDL logic blocks.

When routed through MIO, the TPIU can be configured as a 2,4,8 or 16 bit width. If routed through EMIO the TPIU width can also be configured as 32 bit width, which is the highest throughput.

Additionally, the TPIU has 3 modes of operation:

- Normal — The formatter is enabled and the TRACECTL pin is used to indicate trigger position and detect when no trace is available.
- Continuous — The formatter is enabled but the TRACECTL pin is not used. Instead, the TPA must decode part of the formatter protocol itself to determine trigger position and trace availability. Data is sent at every edge of the clock.
- Bypass — Trace is output without any modification or formatting. Only valid when a single trace source ID is in use, since there is no mechanism to embed source IDs in the stream.

In this project, we utilize the TPIU in continuous mode, as it is the most fitting when routing through EMIO.

The TPIU block also has 2 different IDLE patterns. The first is 0x7FFFFFFF, this is the frame synchronization pattern and enables a TPA or trace decompressor to find the start of a frame. It is output periodically between frames. It is output least significant bit first, starting with bit[0]. In continuous mode, the TPA might discard all frame synchronization packets after the start of a frame is found.

The second pattern is 0x7FFF7FFF, which is the halfword synchronization packet and enables a TPA to detect when the trace port is idle and there is no trace to be captured [2].

Embedded Trace Buffer

The ETB is an on-chip trace sink that receives the ATB trace stream and stores it in a dedicated internal Static Random Access Memory (SRAM). The buffer operates as a circular buffer, meaning that when it reaches capacity it wraps around and overwrites the oldest data, ensuring that the most recent trace is always retained. On the Zynq-7000, the ETB provides 4 KiB of on-chip storage. Due to its limited size, it is typically used in conjunction with triggering and filtering to capture only the trace of interest. In this project, the ETB serves as a secondary sink alongside the TPIU, receiving the same replicated trace stream via the replicator.

Trace Memory Controller

The TMC is a configurable CoreSight trace component designed as a successor to the ETB. Depending on how it is configured at integration time, it can operate in one of three configurations: as an ETB, storing trace in a dedicated on-chip SRAM used as a circular buffer; as an Embedded Trace FIFO (ETF), which uses its SRAM as either a circular buffer or a First in first out (FIFO); or as an Embedded Trace Router (ETR), which routes trace over an AXI bus to system memory or any other AXI slave, enabling much larger trace captures than an on-chip SRAM allows.

The ETF has two modes of operation relevant to this project. In Software FIFO mode it acts as a trace sink, much like the ETB. In Hardware FIFO mode it acts as a link between a trace source and a trace

sink, functioning as a buffer that absorbs bursts in trace bandwidth and smooths them out before they reach the downstream sink, reducing the likelihood of trace overflow. [20]

2.5 CoreSight Frame Format

The ARM CoreSight trace infrastructure transmits data in 16-byte frames. Each frame carries a multiplexed stream of trace data from one or more trace sources, identified by their unique trace IDs. Within a frame, the source ID context is maintained across bytes — once an ID is established, subsequent data bytes are implicitly attributed to that source until a new ID byte is encountered.

As illustrated in Figure 2.2, each frame consists of 15 data-carrying bytes and one auxiliary byte. Every even-numbered byte (bytes 0, 2, 4, ..., 14) is ambiguous: its least-significant bit, called the F bit, determines whether the byte carries a 7-bit source ID or a data payload.

If the F bit is 0, the byte is a data byte, and the lost Least Significant Bit (LSB) is recovered from the corresponding auxiliary bit in byte 15, restoring the full 8-bit data value. If the F bit is 1, the byte is an ID byte, and the corresponding auxiliary bit indicates whether the ID change takes effect immediately or is delayed.

An auxiliary bit of 0 means the new ID applies from the very next byte onward; an auxiliary bit of 1 means the ID change is delayed by one byte, so the byte immediately following the ID byte still belongs to the previous source. All odd-numbered bytes are unconditionally data bytes. Byte 15, the final byte of each frame, is the auxiliary byte. It contains one auxiliary bit per even-numbered byte: bit 0 corresponds to byte 0, bit 1 to byte 2, and so on up to bit 7 corresponding to byte 14.

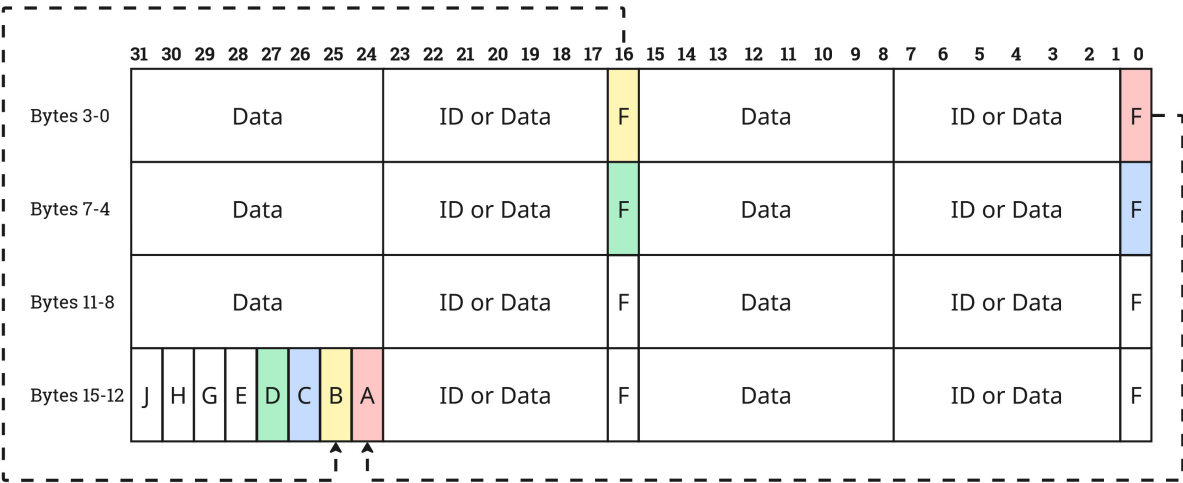


Figure 2.2: Illustration of the CoreSight 16-byte frame format.

Chapter 3

System Design and Architecture

3.1 System Overview

Figure 3.1 demonstrates the complete system developed in this project. The system is built around the Zynq-7000 SoC, which integrates a dual-core Cortex-A9 PS with PL in a single device. Each processor core has a dedicated PTM which feeds trace onto the ATB, where a funnel merges both streams into a single trace stream. A replicator then duplicates this stream to two parallel sinks: the ETB and the TPIU.

Before trace can be captured, the CoreSight components must be configured and enabled. In this project, this is performed in software running on the PS using a combination of Vitis and the CSAL. The full details of this configuration process are described in Section 6.4.

The ETB captures trace into its 4 KiB on-chip circular buffer, which can be read out via its memory-mapped registers by software running on the PS and saved to an SD card. This data is then transferred to a host PC, where OpenCSD is used to deformat the raw CoreSight frames and produce a reference output.

In parallel, the TPIU routes the same trace stream via EMIO directly into the PL fabric in continuous mode. There, a Frame Generator reconstructs complete 16-byte CoreSight frames from the incoming data stream, which are then passed to the hardware Frame Deformatter — the primary contribution of this project.

The deformatted output is buffered and transferred to system memory via AXI DMA, from where it is saved to SD card and sent to the host PC for comparison against the OpenCSD reference output. Since the ETB is limited to 4 KiB of trace storage, the capture window is deliberately constrained such that the total trace generated does not exceed this limit. If the captured program produces more than 4 KiB of trace data, the ETB will overwrite its oldest data while the TPIU continues to capture uninterrupted, making a meaningful comparison between the two outputs impossible.

This system is designed as a development platform. The immediate goal is to validate the hardware

Frame Deformatter by comparing its output against a known-good software reference. However, the platform is also intended to accelerate future work: when the next development stage — a hardware ETM decoder — is implemented, this infrastructure can be reused directly. At that stage, the comparison script would need to be extended to compare decoded PTM output rather than raw deformatted frames, but the trace capture pipeline, the PL processing chain, and the SD card readout flow remain unchanged.

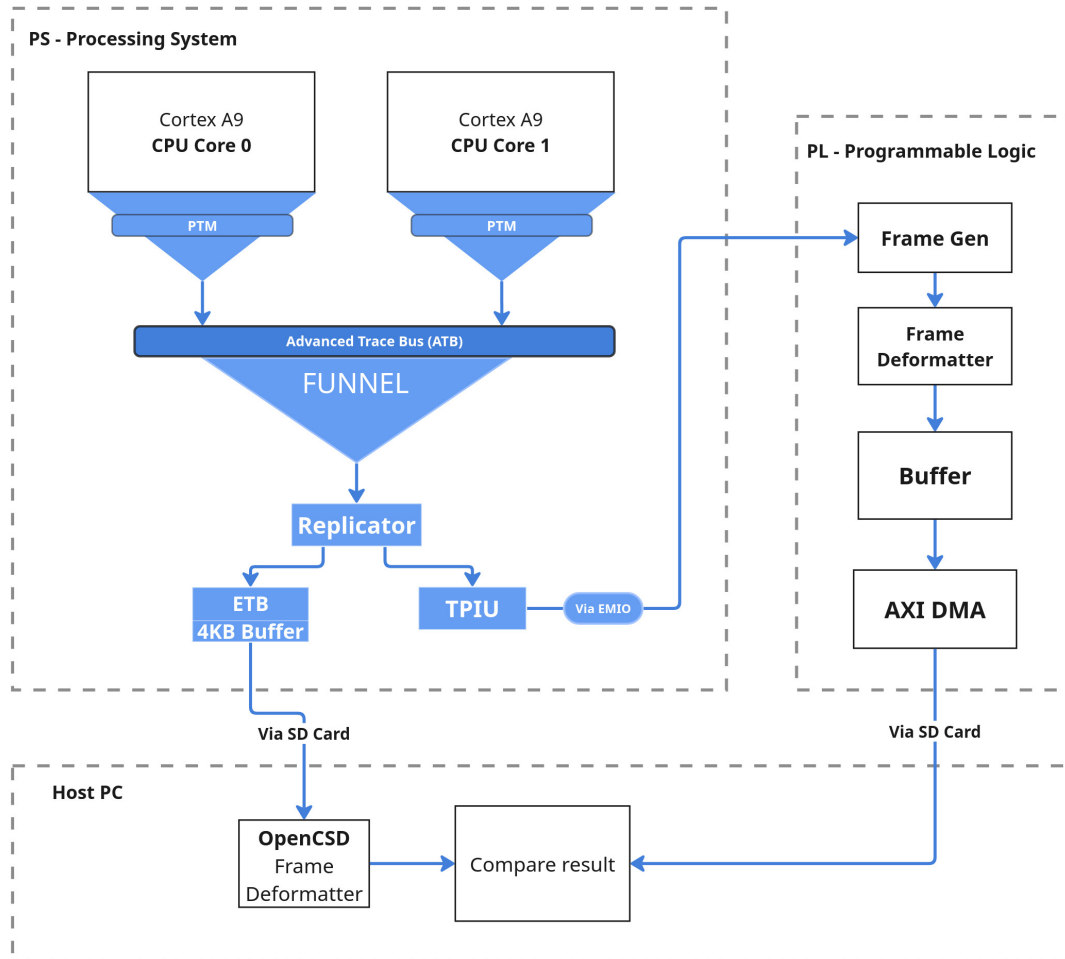


Figure 3.1: Total system for our project

3.2 Tools

3.2.1 Vivado

Vivado is AMD's hardware design suite used in this project for VHDL development, simulation, synthesis, and implementation. Beyond HDL development, Vivado provides a Block design environment where Intellectual Property (IP) cores can be instantiated and connected graphically. Several IP cores are central to this project, including the AXI DMA for transferring deformatted trace data to Double Data Rate (DDR) memory, and the Zynq-7000 PS block which exposes the TPIU interface to the PL. Once the block design is complete, Vivado synthesises and implements the design and generates a bitstream for programming the FPGA fabric [21].

3.2.2 Vitis

Vitis is AMD's software development environment used to write and deploy the C application running on the Cortex-A9 Central Processing Unit (CPU). Since the project requires both FPGA fabric logic and software running on the PS to operate together, the hardware design is exported from Vivado as an Xilinx Support Archive (XSA) file and imported into Vitis. This allows Vitis to program both the bitstream and the C application to the board simultaneously over Joint Test Action Group (JTAG) [22].

3.2.3 Integrated Logic Analyzer

The Integrated Logic Analyzer (ILA) is a Vivado IP core that enables real-time signal monitoring on the FPGA fabric during hardware operation. Probe points are added to signals of interest in the Block design, and the ILA captures their values into an on-chip buffer, which can then be inspected in the Vivado waveform viewer. In this project the ILA was used to observe the live TPIU stream and verify the behaviour of the Finite State Machine (FSM) states and AXI signalling on hardware, complementing the simulation results obtained in Vivado's built-in simulator.

3.2.4 Yocto

Yocto-Project is an open source collaboration project that helps developers create custom Linux-based systems for embedded products, regardless of hardware architecture. It provides a flexible set of tools and a space where embedded developers worldwide can share technologies, software stacks, configurations, and best practices.

Its core components include the OpenEmbedded build system, BitBake as the build engine, and Poky as the reference embedded OS. The Layer Model is its most distinctive feature, allowing developers to separate, share, and reuse collections of build instructions called layers. Recipes define how packages are built, including source locations and dependencies. The build workflow fetches source code, applies patches, compiles, runs QA checks, and generates a final root filesystem image [23].

3.2.5 CoreSight access library (CSAL)

The CSAL, provides an Application Programming Interface (API), which enables user code to interact directly with ARM CoreSight-compliant devices on a target. This allows, for example, the program execution trace to be captured in a production system without the need to have an external debugger connected. The saved trace can be retrieved later and loaded into a debugger for analysis, CSAL can be run on an application core or a management core. The CSAL supports a number of different CoreSight components and has configurations for several target SoCs and boards, and we can modify it to support other CoreSight components [15].

3.2.6 OpenCSD

open source CoreSight Decoding Library (openCSD) is an open source CoreSight trace decode library that provides an API for decoding CoreSight trace streams in three stages. The library is implemented in C++ with an optional C API [24]:

- **Frame Deformatting** removes CoreSight frame formatting from individual trace streams.
- **Packet Processing** separates individual trace streams into discrete packets.
- **Packet Decode** converts packets into fully decoded trace describing the program flow on a core.

In this project, openCSD is used as the software reference implementation against which the hardware deformatter output is validated. openCSD has also been integrated with the standard Linux performance tool `perf`, enabling the configuration of CoreSight components directly from Linux.

3.2.7 Zynq-7000 SoC

The AMD Zynq-7000 SoC is a family that integrates a single or dual-core ARM Cortex-A9 PS with PL on a single 28 nm device. The PS includes a dual-core ARM Cortex-A9 CPU, memory, external interfaces, I/O peripherals, and dedicated CoreSight debug and trace components, while the PL provides the re-configurable FPGA fabric. According to AMD, this architecture combines ASIC-like performance and power efficiency with FPGA flexibility, targeting both cost-sensitive and high-performance applications across automotive, industrial, imaging, wireless, and medical markets [25].

The PS and PL are connected through multiple high-bandwidth AXI-based interfaces, with the PS always booting first and enabling software-controlled PL configuration and dynamic reconfiguration via bitstreams. In this project, the High Performance (HP) AXI interface is used to transfer deformatted trace data from the PL to DDR memory via DMA. PS peripherals can also be routed to the PL via EMIO, which is how the TPIU trace output is brought into the PL fabric in this project [16].

This project targets the ZedBoard, a development board built around the Zynq-7020 (XC7Z020) device, as shown in Figure 3.2. The ZedBoard exposes the necessary EMIO pins and provides the JTAG and SD card interfaces used for programming and trace data retrieval in this project.

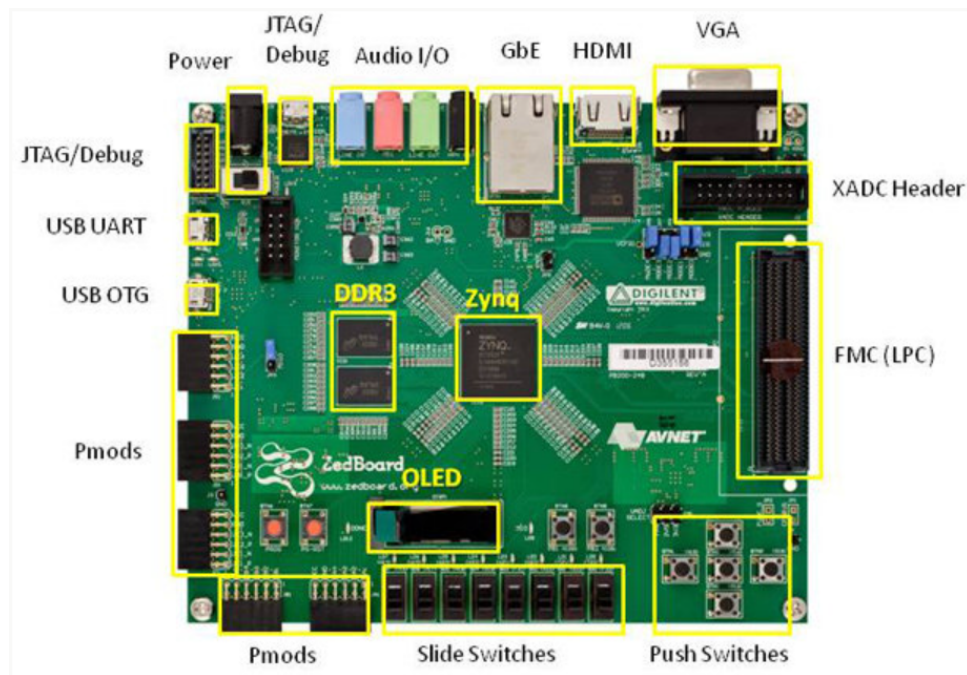


Figure 3.2: AMD Zynq-7000 SoC architecture [1].

3.2.8 Tool Command Language (TCL)

The Tcl, pronounced "tickl" is an open source, cross-platform scripting language created by John K. Ousterhout at the University of California at Berkeley in 1988. Originally conceived as a reusable, embeddable language core for tools being developed by Dr. Ousterhout and his colleagues, it has come to be much more. It is widely used in applications including web scripting, test automation, network/system management, Electronic Design Automation (EDA) and Computer Aided Design (CAD), and in a variety of other fields [26].

3.2.9 Risk Analysis

To systematically identify and manage potential obstacles throughout the project, a risk analysis was conducted at the outset of the development process. Risks were grouped into six categories which reflect the main areas problems could arise in a project of this nature: Technical/Hardware, Data/Overflow, Knowledge/Competence, Project Management, Team/Collaboration, and Resources/Access.

Each identified risk is described by a keyword and a brief description of the failure scenario. Two dimensions are then assessed: the probability of the risk occurring and the consequence it would have on the project if it did. Both are rated on a qualitative scale (Low, Med-low, Medium, Med-high, High), allowing the team to prioritise mitigation efforts toward risks that combine high likelihood with high impact. Finally, a preventive action is defined for each risk, specifying the concrete step taken to reduce either the probability of occurrence or the severity of its impact.

The resulting table (Table D.1) serves as a living reference throughout the project: risks are revisited at key milestones to assess whether the situation has changed and whether additional countermeasures

are needed.

3.2.10 Design Requirements

Prior to implementation, a set of functional and non-functional requirements was defined to guide design decisions and provide measurable criteria for evaluating the final system. The functional requirements capture what the system must do — such as receiving and deformatting CoreSight frames, correctly identifying source IDs, connecting to the TPIU, storing the processed data — while the non-functional requirements address constraints on how it must do so, including timing budget and resource utilisation on the Zynq-7000. The complete requirements are listed in Appendix E.

Chapter 4

Design and VHDL-logic

4.1 Structure

Figure 4.1 illustrates the high-level design of the pipeline. Trace data originates in the PS from the CoreSight trace sources and is transmitted through the TPIU to the Frame Generator, which accumulates incoming 32-bit words until a complete 128-bit frame has been assembled. The completed frame is then passed to the Frame Deformatter, which strips the CoreSight formatting and outputs each data byte alongside its corresponding source ID. Finally, the AXI DMA transfers the deformatted data from the PL to system memory, where it can be retrieved by the host application for further processing.

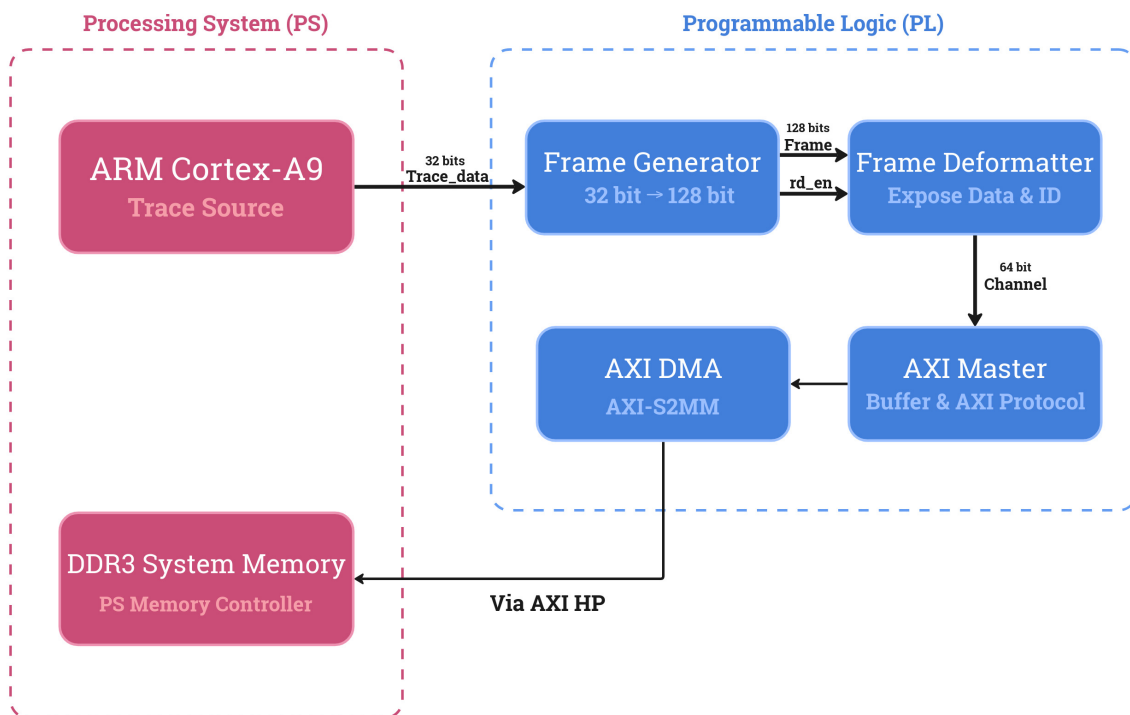


Figure 4.1: CoreSight trace capture pipeline on the Zynq-7000 SoC

4.2 Frame generator

The frame generator is the entry point of the pipeline, responsible for filtering out synchronisation and idle packets from the incoming 32-bit TPIU stream and assembling the remaining data into complete 16-byte frames for the deformatter. A complete frame must be assembled before deformatting can begin, as the 16th byte of each frame contains the auxiliary bits required to correctly interpret every byte preceding it.

Every clock cycle the block receives 32 bits of trace data. If the incoming word is neither a synchronisation word (0x7FFFFFFF) nor an idle half-word (0x7FFF7FFF), the word is shifted into a 128-bit register as shown in Figure 4.2. Once four valid 32-bit words have been accumulated, a full 16-byte frame is ready and `rd_en` is asserted for one cycle to signal the deformatter. The resulting waveform is shown in Figure 5.4.

A basic error check is included: if a synchronisation frame arrives while the internal word counter is non-zero, the error flag is asserted, indicating a synchronisation frame appeared mid-frame. This should not occur in a well-formed TPIU stream and likely points to an alignment issue in the data path.

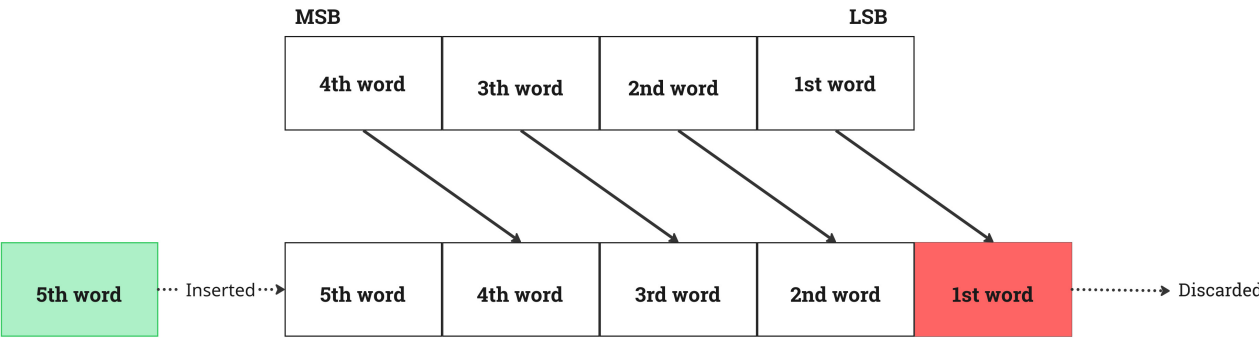


Figure 4.2: Logical right shift.

4.3 Frame Deformatter v1.0

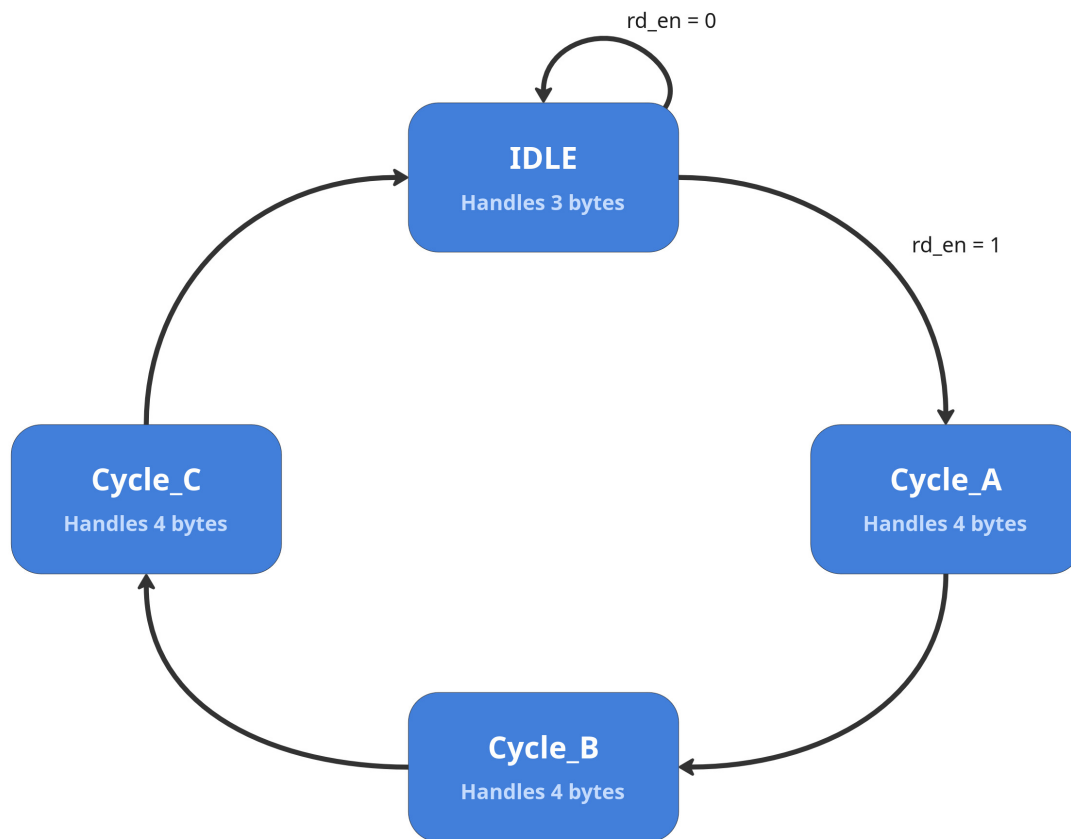


Figure 4.3: State diagram for the deformatter

The frame deformatter is responsible for deformatting the 16-byte frames produced by the CoreSight formatter. Because a new frame arrives every four clock cycles, the decoder must complete its work within exactly four cycles to stay in step with the incoming data. This constraint drives the choice of a four-state FSM: IDLE, Cycle_A, Cycle_B, and Cycle_C, as shown in Figure 4.3.

The deformatter is parallelised, with each state responsible for deformatting 3-4 bytes of data. The IDLE state handles 3 bytes, while the "cycle" states each handle four bytes. IDLE is intentionally given fewer bytes because it must also check the `rd_en` handshake signal from the frame generator and capture a snapshot of the incoming frame into a register. This snapshot, is then used by all subsequent states so that the logic operates on stable register values.

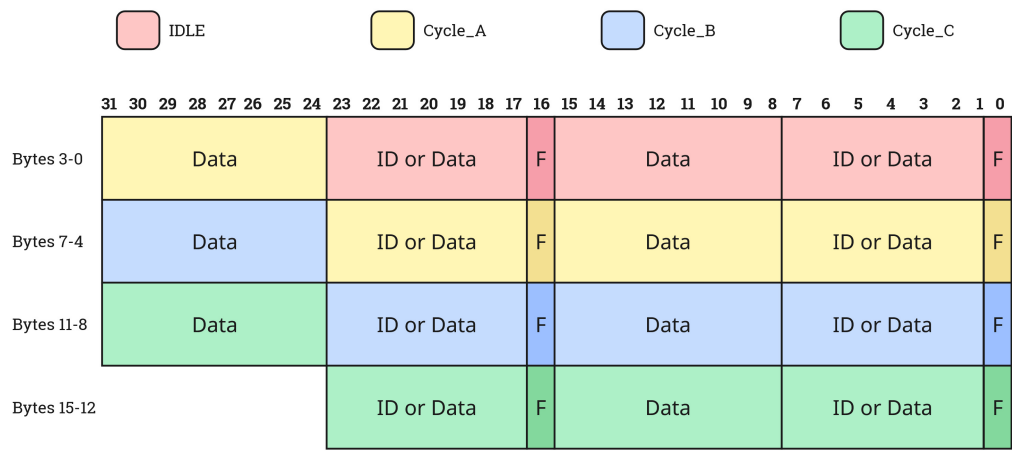


Figure 4.4: Logic split across 4 clock cycles

Distributing the logic across four cycles is a deliberate timing optimisation. By limiting the combinational logic within each clock period, the design can achieve good timing slack and operate at higher clock frequencies than would be possible if the entire frame were decoded in a single cycle. Figure 4.4 illustrates how the frame is partitioned across the four states.

Although the FSM has four states, only two distinct deformatting patterns need to be implemented. That is, the idle state and the cycle states. The cycle states all process exactly four bytes using identical structural logic.

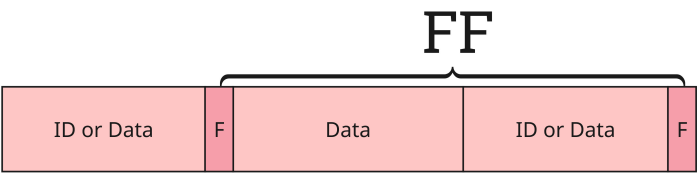


Figure 4.5: F-bit concatenation for case statement F.

As described in Section 2.5, even-numbered bytes carry either a source ID or a data payload determined by their F bit, with odd-numbered bytes being unconditionally data. Each decoding state handles exactly two even-numbered bytes. Their F bits are concatenated into a two-bit selector vector, labelled `v_selector` in the implementation and illustrated in Figure 4.5. All four possible combinations can then be dispatched with a single case statement as shown in Table 4.1, avoiding deeply nested if-else chains and keeping the logic straightforward to synthesise.

FF	Byte 2	Byte 0
00	Data	Data
01	Data	ID
10	ID	Data
11	ID	ID

Table 4.1: The four possible combinations of the `v_selector` vector.

Figure 4.6 shows a concrete example of the selector value 01, meaning that byte 2 is data and byte 0 is an ID. In this case, the ID's auxiliary bit indicates that the ID is delayed. This means byte 1 belongs to the previous source ID, and the new ID only becomes active from byte 2 onward.

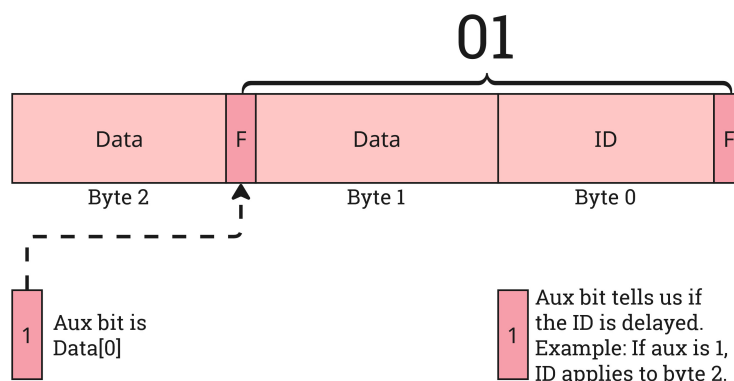
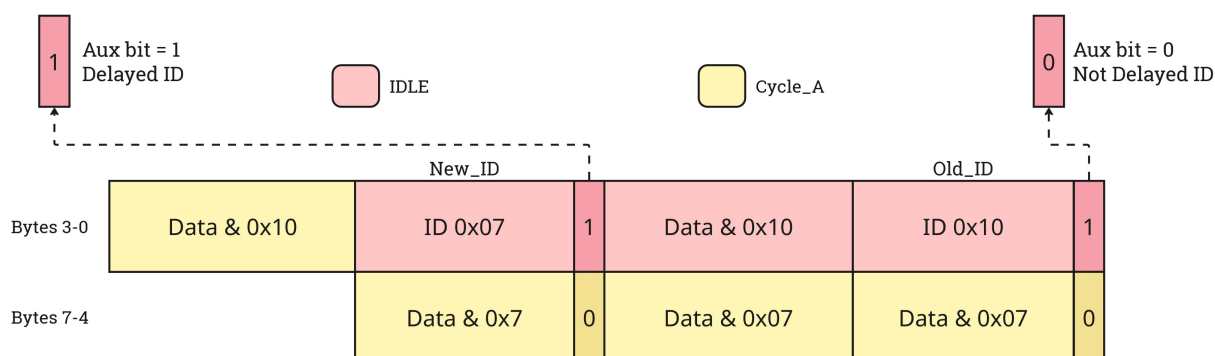


Figure 4.6: Example case "01" – byte 2 is Data, byte 0 is ID

This delay mechanism introduces a state-carrying requirement across clock cycles. At the end of each cycle, the decoder saves the most recently encountered ID into the register `new_id`, along with its auxiliary bit in `NewID_isDelayed`. If that ID was itself preceded by another ID in the same cycle, the earlier ID is additionally saved into `old_id`. At the start of the next cycle, if `NewID_isDelayed` is asserted, the first data byte in that cycle must be tagged with `old_id` rather than `new_id`. After that single byte, `NewID_isDelayed` is cleared and `new_id` becomes the active ID for all following data in the cycle.



Cycle_A uses both ID's from IDLE state. It is necessary we save these ID's into variables. `New_ID` and `Old_ID`, as well as `New_ID`'s aux bit to know if it was delayed.

Figure 4.7: Delayed-ID carry example across IDLE – Cycle_A

Figure 4.7 illustrates this carry mechanism at the boundary between the IDLE state and Cycle_A.

In the example, the IDLE state encounters an ID byte of 0x07 with its auxiliary bit set (delayed ID). The ID value is stored in `new_id` and `NewID_isDelayed` is set to 1. The preceding ID is also stored into `old_id` (0x10). When Cycle_A begins, it detects that `NewID_isDelayed` is asserted and therefore assigns `old_id` (0x10) to the first data byte of the cycle. After that assignment `NewID_isDelayed` is

cleared, and `new_id` is used for all remaining data bytes in the cycle. The same carry logic repeats at every cycle boundary, ensuring correct ID attribution regardless of where ID bytes and delayed transitions fall within the frame.



Figure 4.8: Deformatter output channels

The deformatter produces four output channels per clock cycle — Chan0 through Chan3 — which are concatenated into a full 64-bit vector. For simulation purposes, however, the channels are used directly to improve visibility. As shown in Figure 4.8, each channel is 16 bits wide: eight data bits occupy the most-significant position, seven ID bits occupy the least-significant position, and a single filler bit sits between them.

This filler bit was originally intended as a valid flag, but was dropped because it misaligned the ID field in hex, making it unreadable at a glance in the waveform viewer. Validity is instead signalled implicitly: any channel with an ID field of 0x00 is considered invalid and must be discarded by the downstream logic. Invalid channels are therefore driven to all-zero, which aligns with the CoreSight formatter’s convention that ID 0x00 is reserved and never assigned to a real trace source.

The net result is that the deformatter outputs 64 bits per clock cycle, comprising 32 bits of deformatted trace data labeled with 32 bits of ID, in a compact, fixed-width format that is straightforward to consume by the next stage in the pipeline. By deformatting a frame in exactly four cycles and spreading the combinational logic evenly across them, the design meets its throughput target while remaining synthesisable at high clock speeds.

4.4 Converting to axi

For storing the trace data, AXI was chosen as the transport layer. It is the preferred PS-PL interface in Vivado and has a straightforward DMA IP core available, making it well suited for bulk data transfers from the programmable logic to DDR memory. The AXI4-Stream protocol is a point-to-point streaming interface where a master drives data to a slave. The signals relevant to this implementation are TDATA, which carries the data payload, TVALID, which the master asserts to indicate that the data on TDATA is valid, TREADY, which the slave asserts to indicate it is ready to accept data, and TLAST, which the master asserts on the final beat of a transfer to signal its end. A data transfer occurs on any clock edge where both TVALID and TREADY are asserted simultaneously [27].

The TLAST signal poses a challenge when the input is a continuous TPIU stream, as there is no natural end to the data. To resolve this, the transfer length must be bounded in some way.

In this project, the TPIU output is intended to be compared against the ETB output. The ETB is a circular buffer with a fixed capacity of 4 KiB, meaning any trace data beyond this limit will be over-

written. To ensure the two results remain comparable, the TPIU capture must therefore be bounded to the same amount of trace data. Since the deformatter expands each byte of trace data into a 16-bit labeled sample across four channels, the equivalent TPIU buffer size is 8 KiB.

The solution is a VHDL buffer, sized to 8 KiB and initialised to all-zeroes, which sits between the deformatter and the AXI DMA. The buffer operates as a FSM with three states: FILLING, STREAMING, and DONE. On reset, the buffer enters the FILLING state and begins accepting deformatted trace data from the deformatter. Once the PS wishes to initiate a DMA transfer, it asserts a General Purpose Input/Output (GPIO) signal, which the VHDL logic monitors. On detecting this signal, the buffer transitions to the STREAMING state and begins driving TVALID and TDATA towards the AXI DMA, replaying the captured buffer data word by word. TLAST is asserted on the final word of the buffer, after which the FSM transitions to the DONE state where TLAST is deasserted and TVALID is driven low, leaving the interface idle until the next transfer is initiated.

Because the buffer is pre-filled with zeroes, any portion of the buffer that was not written by the deformatter will be transmitted as zero-valued samples. This is benign: as established in previous sections, any sample with an ID field of 0x00 is considered invalid and discarded by the downstream processing. The zeroes therefore pad the transfer to a fixed length without corrupting the trace data.

The buffer operates as a FSM with three states: FILLING, STREAMING, and DONE, as illustrated in Figure 4.9.

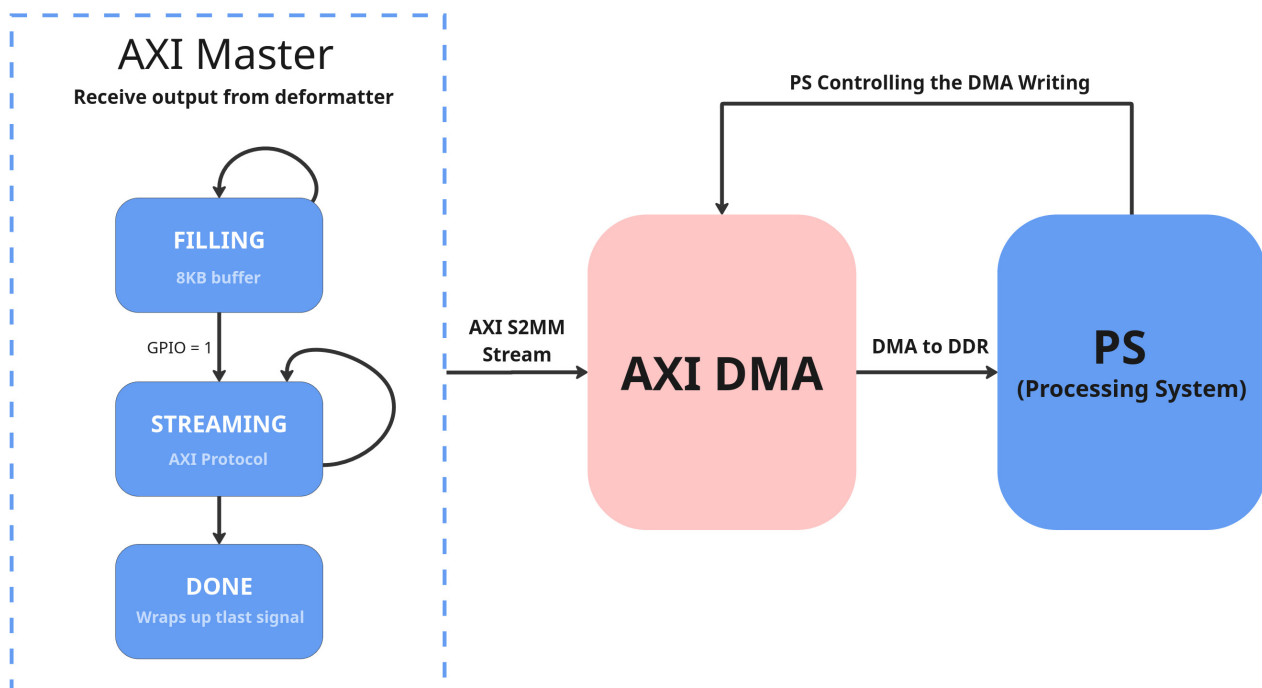


Figure 4.9: AXI Protocol FSM and connections of the AXI network

Chapter 5

Simulation and Verification

5.1 Verification Strategy

Verification was carried out in two phases. First, the *Frame Generator* and *Frame Deformatter* were tested independently using dedicated VHDL testbenches, with a known reference frame from the ARM CoreSight Technical Reference Manual (TRM), [2] serving as ground truth. This allowed functional correctness of each component to be established in isolation before integration.

In the second phase, pre-captured trace data was loaded into Block RAM (BRAM) to simulate a continuous TPIU stream, and the full pipeline was exercised against approximately 25 KB of real trace data. The output was cross-validated against OpenCSD, the open source ARM CoreSight decoding library, using an automated C# comparison script.

5.2 Individual VHDL testbench

5.2.1 Frame Generator Test-1:

The first simulation was conducted for the Frame Generator. During this simulation, a bug was identified in which the 32-bit trace data words were being concatenated into the 128-bit frame register in the wrong order, as illustrated in Figure 5.1.

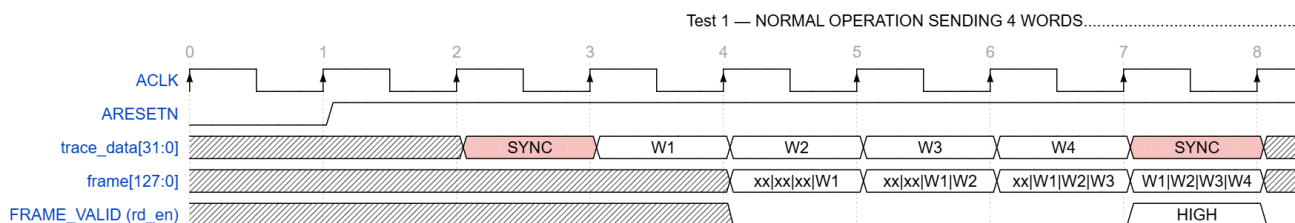


Figure 5.1: Frame Generator testbench output showing incorrect word ordering

The fault was traced to the assignment shown in Figure 5.2, which appends each incoming word to

the least significant end of the register, causing the first word received to occupy the most significant position and reversing the assembled frame. The corrected assignment in Figure 5.3 shifts existing content toward the Most Significant Bit (MSB) and places the incoming word at the LSB, ensuring that byte 0 of the frame resides at the LSB of the frame register as the Frame Deformatter expects. After applying this correction, the testbench output confirmed the expected byte ordering, as shown in Figure 5.4.

```
1 frame_reg <= frame_reg(96
  ↳ downto 0) & trace_data;
```

Figure 5.2: Wrong: appends to LSB, reversing frame order

```
1 frame_reg <= trace_data &
  ↳ frame_reg(127 downto 32);
```

Figure 5.3: Correct: shifts to MSB, preserving frame order

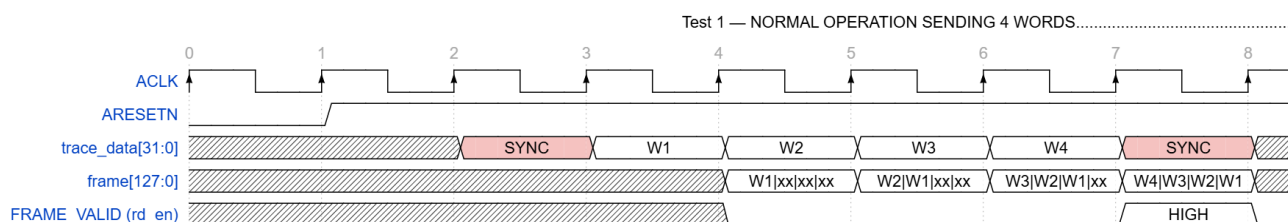


Figure 5.4: Frame Generator testbench output showing correct word ordering

5.2.2 Frame Generator Test-2:

The HALFWORD represents an idle filler pattern and carries no valid frame data. When it appears between valid data words, it must be ignored entirely; the data word counter must not increment and frame assembly continues as though the HALFWORD never occurred. W3 and W4 therefore complete the frame normally, and FRAMEVALID is asserted only once all four valid data words have been collected. The test confirmed that HALFWORD insertion has no effect on frame assembly, as shown in Figure 5.5.

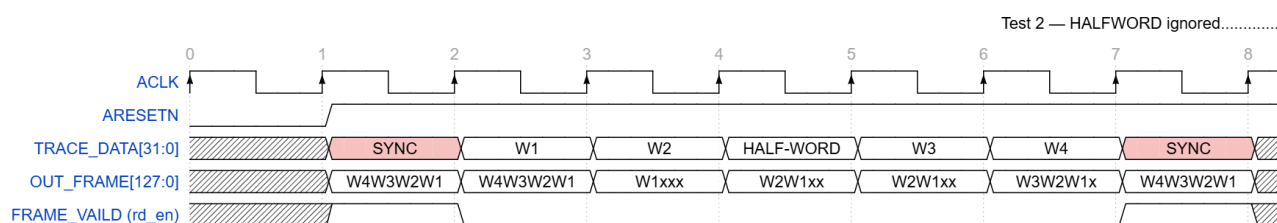


Figure 5.5: Frame Generator testbench output showing HALFWORD IGNORED

5.2.3 Frame Generator Test-3:

In practice, reset is asserted when the design is reprogrammed via Vitis, which clears the PL state and returns all components to their initial conditions. However, the deformatter and frame generator

logic also support mid-trace resets should one be required. When reset is asserted mid-frame, any partially collected data is discarded, the internal word counter and output frame are returned to their initial state, and no FRAMEVALID is generated for the incomplete frame. Once reset is deasserted and a new SYNC arrives, frame collection begins from scratch, as shown in Figure 5.6.

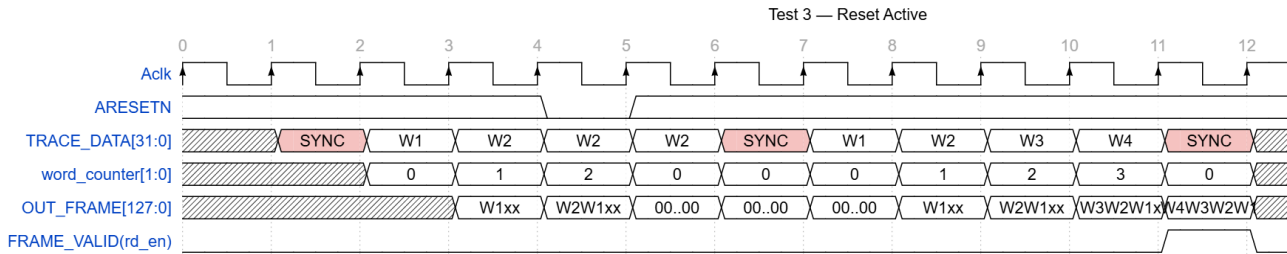


Figure 5.6: Frame Generator testbench output showing RESET ACTIVE

5.2.4 Frame Deformatter test with standard ARM CoreSight Frame

The ARM CoreSight TRM provides a worked example of a complete formatter frame including the expected deformatted output [2]. This was used as the primary test case for the frame deformatter, as it provides an unambiguous ground truth against which the output of our logic could be directly verified. The example frame is shown in Figure 5.7.

	31							24 23							17 16 15							8 7							1 0									
Bytes 3-0	Data 0xA7							Data 0x53							0							Data 0xAA							ID 0x03							1		
Bytes 7-4	Data 0x52							Data 0x2A							0							Data 0xA8							ID 0x15							1		
Bytes 11-8	Data 0xCA							ID 0x03							1							Data 0x54							Data 0x29							0		
Bytes 15-12	0	0	0	1	1	1	0	0	Data 0x64							0							Data 0xC7							Data 0x63							0	

Figure 5.7: Example formatter frame [2]

This frame was applied as input to the deformatter testbench. The resulting waveform is shown in Figure 5.8, where each channel output is expressed as a data byte followed by its corresponding source ID. For example, **AA 03** denotes data byte **0xAA** belonging to source ID **0x03**. Channels with an ID of 0x00 are invalid and are disregarded.

The ID is extracted by reading the bytes at even positions within the frame (byte 0, 2, 4, ..., 14). These bytes contain the ID information according to the defined frame format.

Byte(15) <= (MSB) 1cc8c7c6_ca075452_5254a82b_a7a6aa07 (LSB) => Byte(0)

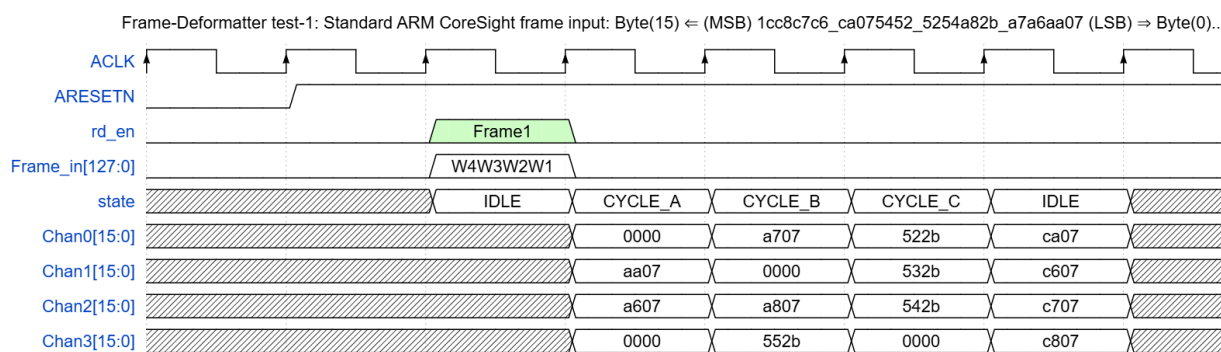


Figure 5.8: Frame deformatter testbench output using the TRM example frame

The channel outputs match the expected values given by the TRM, confirming correct deformatting behaviour for this frame.

5.3 Integration: BRAM Test Infrastructure

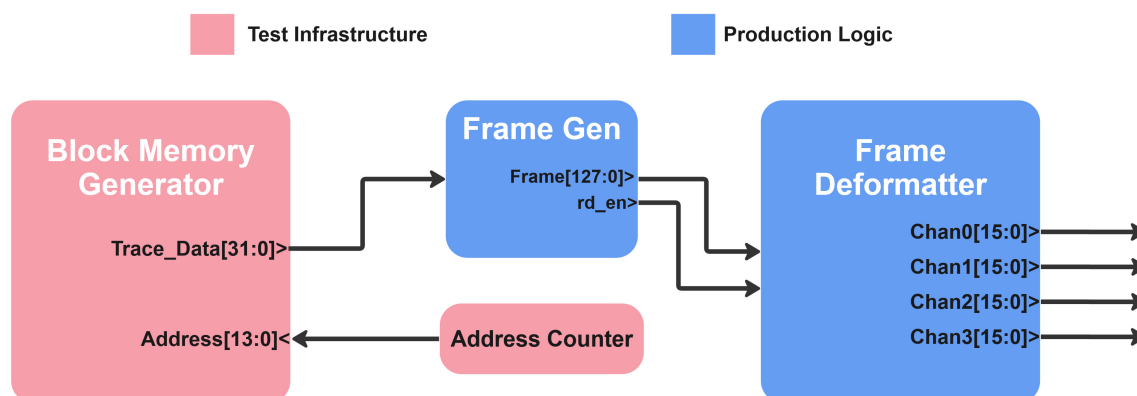


Figure 5.9: Simplified BRAM Dataflow Block Diagram

While the ultimate goal is to connect the deformatter logic directly to a live TPIU stream, doing so requires configuring the ARM CoreSight component registers and introduces non-deterministic trace content that varies between runs, making systematic verification difficult. To address this, three binary files containing trace data captured from a US+ via the ETR trace sink were provided for use in simulation and testing.

These files were converted from binary to Xilinx .coe format using a conversion script, allowing them to be loaded directly into the FPGA's block RAM via the Block Memory Generator IP. Combined with a simple address counter incrementing at every clock cycle, this arrangement effectively simulates a continuous TPIU trace stream, providing a deterministic and fully reproducible input to the deformatter logic. The resulting test infrastructure is illustrated in Figure 5.9. Vivado automatically generates a VHDL testbench when a block design is exported as an HDL wrapper, which was used to simulate the full pipeline against the .coe file contents.

To verify correctness of the data we used openCSD, the open source ARM CoreSight decoding library as a reference implementation. Since the three trace files amount to approximately 25 KB of data, manual comparison of the deformatter output against the openCSD reference was not feasible. A C# script was therefore developed to automate the comparison, providing a reliable and repeatable verification process.

5.3.1 Processor System Reset Hold Time

When simulating the full Block design, all PL logic appeared to be delayed by roughly 700 ns before becoming active. This was traced back to an intentional hold period imposed by the Processor System Reset IP block.

As shown in Figure 5.10, the PS reset signal `ext_reset_in` is not immediately stable on startup, exhibiting multiple glitches before settling [3]. Connecting this signal directly to the PL logic would risk corrupting the design state during initialisation. The Processor System Reset IP filters these glitches and holds all PL resets active until the system has fully stabilised, which accounts for the observed delay.

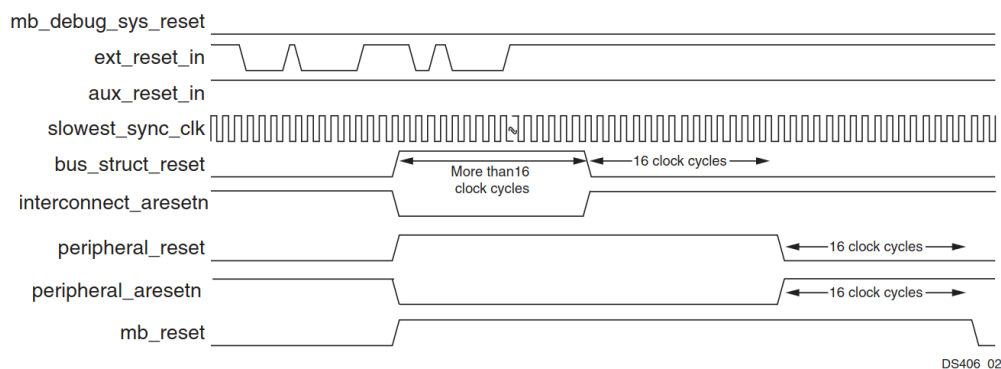


Figure 5.10: PS reset signal fluctuating on startup [3].

5.3.2 BRAM output latency

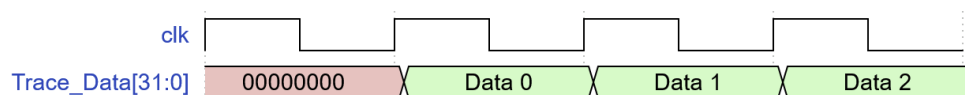


Figure 5.11: Observed 1-cycle delay producing 32 bits of zeroes on first output

Before simulating the full pipeline, it was first necessary to verify that the Block Memory Generator outputs 32 bits of valid data every clock cycle. During this verification, it was observed that the IP core consistently outputs 32 bits of zeroes on the first clock cycle before presenting the actual .coe file contents.

The Block Memory Generator product guide [28] states that the latency of the output varies depending on the output register configuration. Regardless of this setting, the underlying BRAM primitives

on 7 series FPGAs operate synchronously, meaning valid data appears on the output port one clock cycle after the address is presented. On the first rising clock edge the output has therefore not yet been loaded with valid data, resulting in a single cycle of zeroes before the contents of the .coe file appear on the output.

Should these zero bytes propagate into the pipeline, the Frame Generator would incorrectly process them as valid trace data, corrupting the first frame. To circumvent this, a simple two-state finite state machine was added before the Frame Generator, holding the pipeline in an IDLE state for the first clock cycle and transitioning to ACTIVE on the second. This keeps the system modular, so we do not need a separate Frame Generator for the bram simulations. It should be noted that this state machine is specific to the BRAM-based simulation infrastructure and must be removed when interfacing with the live TPIU stream on hardware.

5.3.3 Testing the Whole Design via HDL Wrapper

With the BRAM infrastructure in place, the complete pipeline was exercised end-to-end through the HDL wrapper. The first frame read from the .coe file is 80000180_00000000_00000000_00000021, where byte 0 carries source ID 0x10 and byte 15 contains the auxiliary bits. The deformatter correctly identified the ID byte, consumed the auxiliary byte, and routed the remaining data bytes to the output channels tagged with ID 0x10, with no protocol errors raised. The resulting waveform is shown in Figure 5.12.

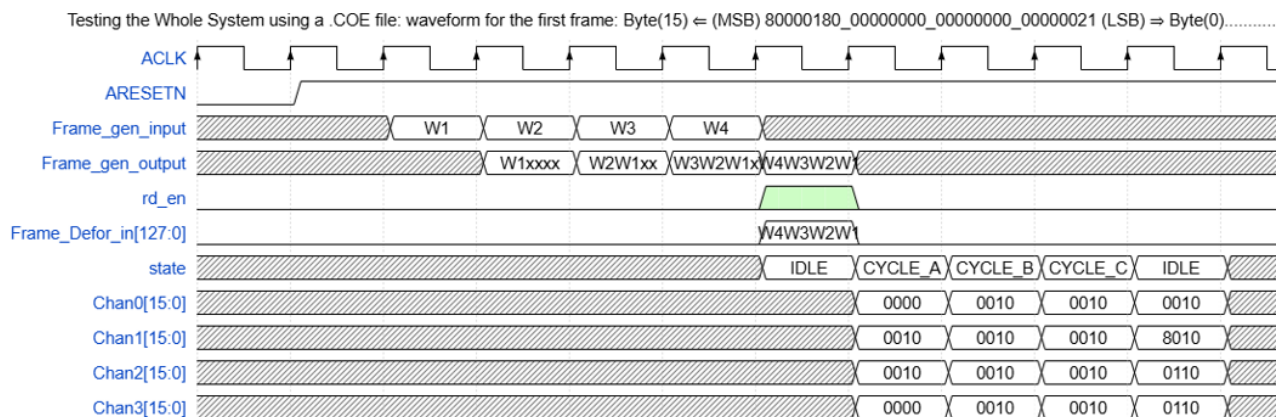


Figure 5.12: Full pipeline simulation using the .coe file.

5.3.4 Full Pipeline Verification Against OpenCSD

To automate the comparison against OpenCSD, three tools work in sequence: a Tcl script captures the deformatter output to CSV, OpenCSD processes the same raw trace data independently as a reference, and a C# script reads both files in parallel and flags any mismatches.

The Tcl script was written for the Vivado simulator. It polls the simulation at 10 ns intervals, begins recording on the first rising edge of `rd_en`, and logs the four output channel values each clock cycle. Each 16-bit channel value encodes the data byte in the upper half and the source ID in the lower half.

At 125 MHz, recording stops when the BRAM address counter reaches 0x908, corresponding to a simulation time of 19,000 ns. This boundary reflects the end of the actual trace data rather than a hard architectural limit, the BRAM is configured with 2337 entries, and any unused entries beyond the end of the trace are padded with zeroes by the block memory generator. This means any compliant .coe file up to approximately 9 KB can be substituted without modifying the block memory generator configuration. An excerpt of the resulting CSV is shown in Figure 5.13.

```
Time_ns, Chan0, Chan1, Chan2, Chan3
675, 0000, 0000, 0000, 0000
685, 0000, 0010, 0010, 0000
695, 0010, 0010, 0010, 0010
705, 0010, 0010, 0010, 0010
715, 0010, 8010, 0110, 0110
```

Figure 5.13: Excerpt of the channel output CSV recorded by the TCL script.

The same .coe file initialized inside the block memory generator was also deformatted through OpenCSD, the first frame of this output is as shown:

```
Frame Data; Index 0; ID_DATA[0x10]; 00 00 00 00 00 00 00 00 00 00 00 80 01 01
```

A C# script then read the CSV and the OpenCSD output files in parallel, extracting the data byte and source ID from each channel entry and comparing them. The trace data given to us also contained STM trace data with the ID 0x20, all entries with that ID and ID 0x00 are skipped as invalid. The reason STM trace is skipped here is simply due to this being an ETM/PTM based project.

As shown below, if a mismatch is detected the script will output the simulation time and opencsd index, allowing you to look it up in the simulation waveform and trace back the error in the VHDL code.

```
Last OpenCSD index: checked 26; Last sample: 770, Mismatch detected.
```

The comparison script reports only on mismatches; a silent run indicates a complete match. The script was run against all three trace files, totalling approximately 25 kB of trace data, with no mismatches reported in any run. This validates that the hardware deformatter produces output identical to the OpenCSD reference across a realistic multi-source trace stream.

5.4 Simulation and Verification Summary

Testing was carried out across four key areas. The Frame Generator was verified through three targeted tests, uncovering and correcting a word-ordering bug in 128-bit frame assembly, confirming correct handling of idle half-word patterns, and validating clean reset behaviour mid-collection.

The Frame Deformatter was validated in isolation using the reference frame from the ARM CoreSight TRM, with output matching the expected values exactly. The BRAM test infrastructure was then

introduced to provide deterministic, reproducible input, where two timing issues were discovered and resolved: a one-cycle zero output from the Block Memory Generator on startup, and a 700 ns reset hold imposed by the Processor System Reset IP. Both were handled by a simple two-state FSM in the Frame Generator.

Finally, the full pipeline was exercised against 25 KB of real interleaved trace data from four ETM sources and compared byte-for-byte against OpenCSD, with all source IDs and data bytes matching exactly. These results confirm the design is functionally correct and ready for integration with a live TPIU stream.

Chapter 6

Hardware Integration and Validation

The previous chapters verified the correctness of the frame deformatter using testbenches and prerecorded trace data loaded into block memory. The final goal, however, is to connect the deformatter to a live TPIU stream originating from the Cortex-A9 processor on the ZedBoard. This chapter describes the steps required to achieve that: configuring the TPIU in Vivado, choosing and configuring a software environment for enabling the CoreSight components, and validating the complete pipeline on hardware.

6.1 Configuring the TPIU in Vivado

Enabling the TPIU begins in the Zynq Processing System IP block in Vivado. Under *MIO Configuration* → *Programmable Logic Test and Debug*, enabling the *Trace* option exposes the TPIU interface. The trace output can be routed either to external pins via MIO, or directly into the PL fabric via EMIO. Since this project processes trace data in VHDL logic on the PL, EMIO routing was selected. The trace width was set to 32 bits, the maximum supported over EMIO, to allow the highest possible throughput.

The clock must also be configured. Under *Clock Configuration* → *System Debug Clocks*, the TPIU clock should be set to the same frequency as FCLK to ensure the TPIU and the PL logic operate on a shared clock domain, in our case we used 125 MHz.

After completing this configuration, a TRACE_0 output port becomes visible on the Zynq PS block. Expanding it reveals four signals. TRACE_CLK_OUT is an internal clock divider intended only for MIO-routed TPIU configurations and can be disregarded here. TRACE_CLK drives the internal TPIU logic and should be connected to the same clock used by the PL. TRACE_CTL carries the trigger signal and is only relevant outside of continuous mode, so it too can be left unconnected for this project. TRACE_DATA [31 : 0] is the 32-bit trace data bus that feeds directly into the frame generator.

With the TPIU connected in the Block design, the hardware interface is complete. However, the CoreSight components themselves, including the TPIU, are disabled by default to conserve power. Activating them requires writing to their memory-mapped control registers in software. Each com-

ponent exposes dozens of registers, making direct register manipulation impractical. Two libraries exist to handle this: OpenCSD and CSAL. This project uses CSAL, as it is ARM's official interface library for CoreSight components [15]. One limitation encountered during the project is that CSAL does not include built-in support for the TPIU component. However, prior work in this area has updated the CSAL library with support for the TPIU, we chose to adapt this fix to our design. [4].

6.2 Software Environment: Linux or Baremetal

Beyond the hardware configuration, a software environment is needed to run the CSAL program that configures and enables the CoreSight components. Two approaches are viable on the Zynq-7000: a full Linux build, typically constructed using Yocto or PetaLinux, or a baremetal application running directly on the Cortex-A9 using Vitis. This project explored both, and the following sections describe the trade-offs between them to help inform the choice for similar projects.

6.2.1 Linux with Yocto

The Linux approach was pursued first, as CSAL is primarily designed for Linux environments. A custom Linux image was built using the Yocto Project, targeting the ZedBoard. CSAL was cross-compiled into the image via a custom BitBake recipe, and a C application was written to configure the CoreSight components, capture trace into the ETB, and save it to a `.bin` file on the SD card.

The Yocto build process itself introduced several non-trivial obstacles. The `meta-xilinx` and `meta-xilinx-tools` layers for Vivado 2025.2 use a branch named `rel-v2025.2` rather than the standard `scarthgap` branch used by the other layers, which is not clearly documented. Additionally, the ZedBoard device tree file had been relocated in this version of the layers, requiring the following override to be added manually to `local.conf`:

```
1 KERNEL_DEVICETREE:forcevariable = "xilinx/zynq-zed.dtb"
```

Because the ZedBoard is a well-established platform, the device tree already included CoreSight component entries, which simplified kernel configuration. Enabling the CoreSight drivers required only a short `coresight.cfg` kernel fragment added via a `bbappend` recipe.

The Linux approach has meaningful advantages for a production or long-term deployment scenario. OpenCSD integrates directly with the Linux `perf` subsystem, which could allow comparison of trace output without any host PC involvement. AXI DMA integration, while more complex than in baremetal, is well-supported through `udmabuf`. For a system intended to run continuously and autonomously, Linux is the more appropriate foundation.

For this project, however, the Linux build served primarily as a proof of concept to validate the CSAL configuration before committing to a development platform. Once correctness was confirmed, the team transitioned to the baremetal approach described below.

6.2.2 Baremetal with Vitis

The baremetal approach runs a standalone C application directly on the Cortex-A9 using AMD Vitis, without any operating system. Setting up a Vitis project requires three components: a platform component loaded with the .xsa hardware export from Vivado, a static library component containing the CSAL source and header files, and an application component hosting the main C program.

Once correctly configured, the baremetal approach offered considerably faster iteration than the Yocto build. The board can be programmed directly over JTAG without moving an SD card, and Vitis provides first-party support for Xilinx IP drivers including AXI DMA, which simplified integration substantially compared to what would be required under Linux. For a project focused on developing and validating VHDL logic, the ability to modify, deploy, and test quickly is a significant advantage.

One limitation is that OpenCSD does not support baremetal environments, so the decoded reference output cannot be produced on-device. Instead, ETB trace data is saved to an SD card and decoded on a host PC using OpenCSD, then compared against the hardware deformatter output using the C# comparison script described in Section 5.

	Linux (Yocto)	Baremetal (Vitis)
CSAL support	Full	Experimental
OpenCSD support	Yes	No
AXI DMA integration	Complex (udmabuf)	Simple (Xilinx driver)
Programming speed	Slow (SD card)	Fast (JTAG)
Setup complexity	High	Low
Suitable for	Production systems	Development and validation

Table 6.1: Summary of the key trade-offs between linux and baremetal.

6.3 ROM table Component addresses

Before configuring the CoreSight components with CSAL, the base address of each component must be known. These can be found in the platform's Technical Reference Manual, or, when running Linux, by using the `cs1s` utility bundled with CSAL. `cs1s` traverses the ROM table and reports the type and base address of every discovered CoreSight component. Figure 6.1 demonstrate the output obtained from running `cs1s` on the ZedBoard Linux build.

```

1 root@zedboard-zynq7:~# csls -romaddr 0xF8800000
2 ** CSLS: listing CoreSight config...
3 ** CSLS: Using custom ROM address 0xF8800000
4 F889D000: type= 1 - CPU 0.0 SOURCE PTMv1.0
5 F889C000: type= 1 - CPU 0.0 SOURCE PTMv1.0
6 F8899000: type=10 - CTI
7 F8898000: type=10 - CTI
8 F8893000: type=12 - CPU 0.0 PMU (6 counters) auth=CC
9 F8892000: type=11 - CPU 0.0 DEBUG v7 (full CP14) (4 wpt) (6 bkpt);
   ↳ Secure, running auth=FF
10 F8891000: type=12 - CPU 0.0 PMU (6 counters) auth=CC
11 F8890000: type=11 - CPU 0.0 DEBUG v7 (full CP14) (4 wpt) (6 bkpt);
   ↳ Secure, running auth=FF
12 F880C000: type= 2 - SOURCE SWSTIM(0)
13 F880B000: type= 0 - SOURCE
14 F880A000: type=10 - CTI
15 F8809000: type=10 - CTI
16 F8805000: type= 2 - SOURCE SWSTIM(32)
17 F8804000: type= 4 - LINK [FUNNEL: 8 in ports]
18 F8803000: type= 8 - SINK PORT [TPIU]
19 F8802000: type=10 - CTI
20 F8801000: type= 7 - SINK BUFFER(4K) [ETB]
21 ** CSLS: done listing CoreSight config
22 root@zedboard-zynq7:~#

```

Figure 6.1: CSLS script crawls through the ROM table to discover device addresses.

6.4 Configuring the CoreSight Components

With the component addresses confirmed, the CSAL program registers each component using `cs_device_get()`, establishes the ATB topology by connecting sources to links and sinks via `cs_atb_register()`, and configures each PTM with its trace ID and desired capture events. The CSAL setup was adapted from [29]. The topology implemented in this project routes PTM trace streams through the funnel to both the ETB and the TPIU via a replicator, as illustrated in Figure 3.1. An overview of the Vitis application structure is shown in Figure 6.2.

We first attempted initializing the DMA at the start of the script, however this ended up causing the cpu to do work which generated trace. The trace exceeded the 4 KB ETB buffer, because of this we chose to instead handle all DMA logic after the tracing session was completed.

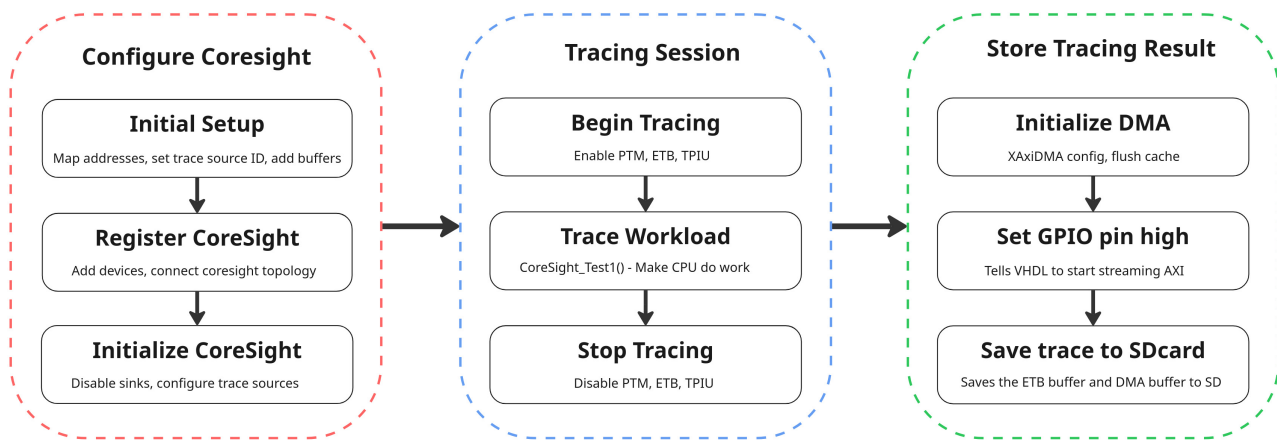


Figure 6.2: Vitis application block diagram

6.5 Hardware Validation Results

With the full pipeline deployed on the ZedBoard, the ILA was used to observe the live TPIU stream and verify correct behaviour of the hardware logic.

6.5.1 TPIU Pre-Enable Behaviour

Before the CoreSight components are enabled by writing to their control registers via CSAL, the TPIU was observed to continuously output the value `0x00000001`, we did not find this mentioned anywhere in the TRM. Under the frame format defined in Section 2.5, this resolves to a source ID of `0x00` followed by data bytes. Since ID `0x00` is reserved and all samples carrying it are discarded by the decoder, this output is harmless in a production system. However, because the hardware buffer only captures the first 8 KB of data passing through the decoder pipeline, the pre-enable output was filling the entire buffer with zeroes before any real trace arrived, leaving no valid data in the DMA buffer.

To resolve this, a latch was added to the Frame Generator. The latch holds the pipeline in an inactive state and does not pass any data downstream until the first synchronisation word (`0x7FFFFFFF`) is observed. Synchronisation words are only emitted once the TPIU has been enabled, so the latch effectively gates the pipeline on the enable event without requiring any explicit software handshake. This ensures that the capture window begins with real trace data rather than pre-enable filler.

6.5.2 Observed Stream Characteristics

During live capture, the ratio of idle words to data words was significantly higher than expected. In practice, a single data word was frequently followed by hundred of idle half-word patterns (`0x7FFF7FFF`) before the next data word arrived. This reflects the bursty nature of real PTM trace output, where the processor may execute long sequences of linear instructions that produce no trace waypoints at all. The Frame Generator correctly filtered all idle and synchronisation patterns and assembled only

the valid data words into frames, consistent with the behaviour verified in simulation.

The deformatter pipeline operated correctly on the live stream. Downstream of the deformatter, the AXI master buffer was observed via the ILA to fill correctly with deformatted channel data. The GPIO signal asserted by the PS was confirmed to transition the buffer FSM from the FILLING state to the STREAMING state as intended, after which the buffer replayed its contents into the AXI DMA IP without error. The ETB capture also functioned correctly, with the CSAL application successfully saving the buffer contents to a .bin file on the SD card.

6.5.3 End-to-End Comparison

A full end-to-end cross-validation between the TPIU hardware deformatter output and the ETB software reference was not completed. While the AXI DMA transfer had previously been confirmed to function correctly and produce valid output files, when we tried cleaning up the project of old files and prepare it for submission, something in the VITIS configuration seems to have been corrupted. The symptom was a DMA output buffer filled with a repeating 0x01 0x00 pattern rather than valid deformatted trace. ILA inspection confirmed that all the VHDL logic itself remained correct, and was producing the expected channel data, localising the fault to the software layer. The issue could not be diagnosed and resolved within the remaining project timeline.

As a result, the end-to-end comparison described in Section 5 was performed against the BRAM-based simulation however we could not do the same with the live TPIU stream. The hardware evidence gathered — correct TPIU framing, correct idle filtering, correct AXI master FSM transitions, and a functioning ETB capture — provides strong confidence that the full pipeline would produce a matching result once the Vitis configuration issue is resolved.

6.6 Implementation reports

6.6.1 Timing Report

The design was implemented and tested on the ZedBoard at the upper frequency of 125 MHz. Table 6.2 presents the Vivado timing summary for both the TPIU system and the BRAM-based simulation infrastructure, showing that all timing constraints are met with positive slack across setup, hold, and pulse width metrics. The total number of timing endpoints is 8424, with zero failures.

Metric	Setup (WNS)	Hold (WHS)	Pulse Width (WPWS)
TPIU system	0.490 ns	0.009 ns	2.750 ns
BRAM System	3.257 ns	0.084 ns	3.020 ns

Table 6.2: Vivado Timing Summary at 125 MHz

Vivado's timing analysis also allows inspection of the critical path. As shown in Table 6.3, at 100 MHz

the longest path has a total delay of 7.376 ns. Notably, the combinational logic itself contributes only 2.316 ns of this delay, with the remaining 4.366 ns attributable to routing.

However, when the clock constraint is tightened to 125 MHz, the logic delay increases while the net delay decreases compared to the 100 MHz case. This is expected behaviour from Vivado's timing-driven place and route: with a stricter timing requirement of 8.0 ns, the tool places logic differently to try to meet the tighter deadline, which changes the balance between logic and routing delay. Regardless, both paths meet their respective timing requirements with slack to spare.

Total Delay	Logic Delay	Net Delay	Requirement
7.376 ns	2.316 ns	5.060 ns	10.0 ns
7.352 ns	3.886 ns	3.466 ns	8.0 ns

Table 6.3: Longest timing path

6.6.2 Utilization report

Table 6.4 presents the FPGA resource utilization for the full TPIU system as reported by Vivado after implementation on the ZedBoard. The design makes modest use of the available resources across all categories, with slice LUT utilization at 4.07% and slice register utilization at 3.11%. The highest relative utilization is seen in slices at 7.19%, which is still well within acceptable limits.

Module	Slice LUTs	Slice Registers	Slices	LUT as Logic	Block RAM Tiles
Total available	53200	106400	13300	53200	140
Resources used	2164	3313	956	2069	6.5
Utilization (%)	4.07%	3.11%	7.19%	3.89%	4.64%

Table 6.4: Vivado Resource Utilization Summary on Zedboard

The low utilization figures confirm that the design leaves substantial room for the subsequent pipeline stages described in Section 1.1.

Previous work

Table 6.5 compares this work against three prior hardware CoreSight decoder implementations. While each implementation may have higher throughput occasionally, the table, which is adapted from Weingarten's comparison [4], only accounts for the worst-case scenarios.

Zeinabolin [30] targets ETMv4 on a xc6vcx75t-2ff784, achieving 125 MB/s at 1 byte/cycle and 125 MHz, primarily for debugging and verification purposes where occasional data loss from buffer overflow is acceptable. Schmid [31] targets PTM on a Zynq-7000 for online runtime verification using TeSSLa specifications embedded on the FPGA, also at 1 byte/cycle and 125 MHz. In Schmid's case, asynchronous FIFOs are an architectural necessity to bridge the gap between the TPIU output domain

and the slower downstream TeSSLa network running at 50 MHz. Weingarten [4] targets ETMv4 on a Zynq UltraScale+ for datacenter profiling, achieving 1 GB/s through loop unrolling at 4 bytes/cycle and 250 MHz, and where the faster UltraScale+ fabric makes it possible to sustain full TPIU throughput without a buffer.

Decoder	Device	Specification	bytes/cycle	Max frequency	Throughput	B
Weingarten [4]	xczu5ev-sfvc784-2-i	ETMv4	4 bytes/cycle	250 MHz	1 GB/s	×
Zeinabolin [30]	xc6vcx75t-2ff784	ETMv4	1 byte/cycle	125MHz	125 MB/s	✓
Schmid [31]	xc7z04	PTM	1 byte/cycle	125Mhz	125MB/s	✓
This work	xc7z020clg484-1	Deformatter	4 bytes/cycle	125 MHz	500 MB/s	×

Table 6.5: Comparison of decoders, with only the worst-case scenario for byte/cycle. B stands for buffer based. Adapted from [4].

This work shares Weingarten's bufferless philosophy, but for a different reason: in a CFI enforcement context, a missing or corrupted instruction packet could cause the analyser to raise a false positive, making buffer overflow a critical failure mode rather than an acceptable trade-off. By processing 4 bytes per cycle in parallel we are able to match the TPIU's maximum port width of 32 bits with no idle cycles. High throughput is therefore not merely a performance goal, but a correctness requirement. The timing implications of processing four bytes per cycle on the Zynq-7000 fabric are discussed in Chapter 7.

Chapter 7

Discussion

7.1 Achievement of design goals

This section evaluates the project outcomes against the functional and non-functional requirements defined in Appendix E.

The core functional requirements were all met. The system was successfully initialised and tested with predefined trace data from memory (F1), forming the basis of the simulation pipeline and enabling isolated verification before any live hardware was involved. Connection to the Cortex-A9 TPIU (F2) was confirmed on the ZedBoard, with correct idle filtering, synchronisation gating, and AXI master FSM transitions observed under live trace conditions. Frame synchronisation (F3) and stream separation (F4) were both fully achieved: the deformatter correctly identifies and discards synchronisation sequences, differentiates between ID and data bytes, and separates trace data by source ID. Byte-for-byte agreement with the openCSD reference library over approximately 25 KB of real multi-source trace data provides high confidence in the correctness of this logic.

Storage of deformatted data (F5) was partially met. Output is captured and accessible via a TCL script during simulation, which was sufficient for verification purposes. On hardware, however, the end-to-end path through the AXI DMA to main memory could not be fully validated: a Vitis configuration corruption late in the project prevented a complete live comparison. ILA inspection confirmed that the PL-side pipeline continued to produce correctly structured channel data throughout, so the gap is confined to the final transfer stage rather than the deformatting logic itself. This situation could have been avoided with more disciplined version control of the Vitis project, which would have allowed the last known-good configuration to be recovered rather than lost.

The non-functional requirements were likewise satisfied. The design is decomposed into distinct, independently testable modules (NF1), the entire pipeline was thoroughly verified in simulation before hardware deployment (NF2), and resource utilisation remained well within the budget of the Zynq-7000 as detailed in Section 6.6.2 (NF3). Code and architecture documentation sufficient to support continued development by others has been provided throughout this report and in the appendices

(NF4).

A notable source of unplanned delay was the learning curve associated with the Xilinx toolchain. Compounding this, AMD recently released a substantially redesigned version of the Vitis IDE, rendering most available documentation and tutorials outdated. Navigating these changes consumed considerable development time that had not been anticipated in the project schedule, directly contributing to the time pressure that led to the version control lapse described above.

7.2 Design decisions and their trade-offs

The most consequential architectural decision was parallelising deformatting across four clock cycles to match the TPIU's 32-bit output without back-pressure. As Table 6.5 shows, prior work such as Zeinabolin and Schmid processes only one byte per cycle, accepting buffer overflow as a recoverable event. In a CFI context this is undesirable, as a missed instruction packet could cause either a missed violation or a false positive.

Distributing logic across four states also had favourable timing consequences. At 100 MHz, logic delay contributes only 2.316 ns of a 7.376 ns critical path, with routing dominating at 5.060 ns. Both constraints are met with slack to spare at 125 MHz, whereas a single-cycle design would almost certainly preclude operation at this frequency on the Zynq-7000 fabric.

The 16-bit output channel format, comprising eight data bits, one filler bit, and seven ID bits, was chosen for waveform-viewer readability. A valid flag in place of the filler bit would prevent manual waveform inspection, which proved useful during development. Using ID 0x00 as an implicit validity indicator is consistent with the CoreSight specification, and the stage 2 pipeline can discard these packets with ease.

The synchronisation latch added to the Frame Generator was not part of the original design, but emerged from an unexpected hardware behaviour: the TPIU continuously outputs 0x00000001 before being enabled by CSAL, filling the capture buffer with zeroed samples before any real trace arrives. The latch gates the pipeline on the first 0x7FFFFFFF synchronisation word without requiring any software handshake. Without it, setting a trigger for the ILA was not possible, and the 4 KiB capture window could not be reliably transferred via AXI DMA. Once stage 3 is complete and trace capture is no longer constrained to a fixed development window, the latch can safely be removed.

7.3 Future Work

The immediate next step is implementing the ETM decoder, the second stage of the three-stage pipeline described in Section 1.1. The deformatter's 64-bit output provides a clean streaming interface for this stage, which must parse variable-length packets from the source-tagged byte stream and reconstruct the sequence of executed branch targets for comparison against the pre-computed CFG in the third stage.

The ZedBoard was provided specifically as a development platform for the deformatter stage, and it has served that purpose well. The validation infrastructure built around it, comprising the BRAM-based simulation harness, the C# comparison script against openCSD, and the Vitis baremetal application, is not board-specific and can be carried over to other platforms without significant rework. Since the CoreSight frame format is identical regardless of the trace source, the VHDL design itself is equally portable and can be reused on any CoreSight-equipped platform without modification.

For subsequent pipeline stages, we recommend migrating to a faster FPGA platform such as the Zynq UltraScale+ MPSoC. As shown in Table 6.2, routing accounts for 5.060 ns of the critical path delay versus only 2.316 ns of logic delay at 100 MHz, making the Zynq-7000 PL fabric the primary timing bottleneck. The additional combinational logic required by the ETM decoder and trace analyser will make timing closure increasingly demanding, so addressing this early is advisable. If migrating to a faster platform is not feasible, a buffer-based approach may offer a practical alternative: under normal operation we observed the TPIU sending dozens to hundreds of IDLE packets between trace words, which could provide sufficient slack for a staged design. That said, for cybersecurity applications in critical domains such as defence, medical, and financial systems, we argue that the added cost of a higher-performance FPGA, one capable of keeping pace with the TPIU stream in real time, is well justified. A newer platform would also unlock access to more recent CoreSight components, such as the Embedded Trace FIFO, which can be interposed between trace sources and sinks to absorb burst traffic and provide an additional safeguard against trace buffer overflow.

Chapter 8

Conclusion

This report has presented the design, implementation and validation of first stage decoding for ARM CoreSight targeting Zynq-7000 FPGA platform. A VHDL pipeline consisting of a Frame Generator and Frame Deformatter was developed, capable of processing live 32-bit TPIU trace streams directly in programmable logic and extracting deformatted data bytes with their corresponding source IDs.

The design was thoroughly validated in simulation using a BRAM-based test infrastructure and verified against openCSD, confirming correct behaviour across a variety of real-world trace.

Hardware integration involved configuring the CoreSight topology using CSAL, with both Yocto-Linux and Vitis-baremetal environments explored, and the Vitis-baremetal approach adopted as the preferred development platform for its faster iteration and simpler AXI DMA integration. Hardware validation on the ZedBoard confirmed correct TPIU framing, idle filtering, and AXI master FSM behavior via ILA observation, with a late-stage Vitis configuration issue being the only obstacle to a complete end-to-end hardware comparison.

The project has therefore achieved its primary objective of demonstrating correct CoreSight deformatting in Programmable logic. The natural next step is resolving the remaining Vitis issue to complete the live hardware verification and then implement an ETM decoder and trace analyzer as a part of the pipeline.

Bibliography

- [1] J.-L. Aufranc (CNXSoft). Zedboard xilinx zynq-7000 community board is now available - CNX software. [Online]. Available: <https://www.cnx-software.com/2012/06/02/zedboard-xilinx-zynq-7000-community-board-is-now-available/>
- [2] Arm CoreSight architecture specification v3.0. [Online]. Available: <https://developer.arm.com/documentation/ih0029/latest>
- [3] Processor system reset module v5.0 product guide (PG164) • viewer • AMD technical information portal. [Online]. Available: <https://docs.amd.com/v/u/en-US/pg164-proc-sys-reset>
- [4] M. E. Weingarten, N. Hossle, and T. Roscoe, “High throughput hardware accelerated CoreSight trace decoding,” in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1–6, ISSN: 1558-1101. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10546666>
- [5] F. Siddiqui, M. Hagan, and S. Sezer, “Establishing cyber resilience in embedded systems for securing next-generation critical infrastructure,” in *2019 32nd IEEE International System-on-Chip Conference (SOCC)*, pp. 218–223, ISSN: 2164-1706. [Online]. Available: <https://ieeexplore.ieee.org/document/9088055>
- [6] M. Abadi, M. Budiu, P. Erlingsson, and J. Ligatti, “Control-flow integrity principles, implementations, and applications,” vol. 13, no. 1, pp. 4:1–4:40. [Online]. Available: <https://dl.acm.org/doi/10.1145/1609956.1609960>
- [7] J. Pincus and B. Baker, “Beyond stack smashing: recent advances in exploiting buffer overruns,” vol. 2, no. 4, pp. 20–27. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/1324594>
- [8] H. Shacham, M. Page, B. Pfaff, E.-J. Goh, N. Modadugu, and D. Boneh, “On the effectiveness of address-space randomization,” in *Proceedings of the 11th ACM conference on Computer and communications security*, ser. CCS '04. Association for Computing Machinery, pp. 298–307. [Online]. Available: <https://dl.acm.org/doi/10.1145/1030083.1030124>

- [9] J. Wilander and M. Kamkar, "A comparison of publicly available tools for dynamic buffer overflow prevention."
- [10] R. Mijat, "Better trace for better software." [Online]. Available: https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/Better_Trace_for_Better_Software_-_CoreSight_STM_with_LTTng_-_19th_October_2010.pdf
- [11] "Arm CoreSight architecture specification." [Online]. Available: <https://developer.arm.com/documentation/ih0029/latest>
- [12] Y. Lee, J. Lee, I. Heo, D. Hwang, and Y. Paek, "Using CoreSight PTM to integrate CRA monitoring IPs in an ARM-based SoC," vol. 22, no. 3, pp. 52:1–52:25. [Online]. Available: <https://dl.acm.org/doi/10.1145/3035965>
- [13] Campus de rennes \textbar CentraleSupélec. [Online]. Available: <https://www.centralesupelec.fr/campus/rennes>
- [14] M. E. Weingarten, "Hardware accelerated trace analysis for compiler optimizations," p. 88 p., artwork Size: 88 p. Medium: application/pdf. [Online]. Available: <http://hdl.handle.net/20.500.11850/612599>
- [15] "ARM-software/CSAL," original-date: 2016-12-14T09:55:31Z. [Online]. Available: <https://github.com/ARM-software/CSAL>
- [16] "Zynq-7000 all programmable SoC technical reference manual (UG585)."
- [17] CoreSight components technical reference manual. [Online]. Available: <https://developer.arm.com/documentation/ddi0314/h>
- [18] CoreSight PTM-a9 technical reference manual r1p0. [Online]. Available: <https://developer.arm.com/documentation/ddi0401/c>
- [19] L. D. Tools, "Training arm CoreSight ETM tracing."
- [20] "CoreSight trace memory controller technical reference manual." [Online]. Available: <https://documentation-service.arm.com/static/5f1060dd0daa596235e7ab7a%3Ftoken%3D>
- [21] Vivado design suite overview • vivado design suite user guide: Getting started (UG910) • AMD technical information portal. [Online]. Available: <https://docs.amd.com/r/en-US/ug910-vivado-getting-started/Vivado-Design-Suite-Overview>
- [22] Getting started with vitis • vitis unified software platform documentation: Embedded software development (UG1400) • AMD technical information portal. [Online]. Available: <https://docs.amd.com/r/en-US/ug1400-vitis-embedded/Getting-Started-with-Vitis>
- [23] Technical overview. [Online]. Available: <https://www.yoctoproject.org/development/technical-overview/>

- [24] “Linaro/OpenCSD,” original-date: 2015-06-16T08:42:58Z. [Online]. Available: <https://github.com/Linaro/OpenCSD>
- [25] “Zynq-7000 SoC data sheet: Overview (DS190).”
- [26] The tcl language. [Online]. Available: <https://tcl.apache.org/html/tcl.html>
- [27] “AMBA AXI protocol specification.”
- [28] Xilinx block memory generator IP core product guide. [Online]. Available: <https://studylib.net/doc/25865054/pg058-blk-mem-gen-2->
- [29] What tooling exists in vitis for interfacing with coresight on a ZynqMP? [Online]. Available: https://adaptivesupport.amd.com/s/question/0D54U00008OIknSSAT/what-tooling-exists-in-vitis-for-interfacing-with-coresight-on-a-zynqmp?language=en_US
- [30] S. M. A. Zeinolabedin, J. Partzsch, and C. Mayr, “Real-time hardware implementation of ARM CoreSight trace decoder,” vol. 38, no. 1, pp. 69–77. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9115651>
- [31] Runtime verification with TeSSLa on enzian. [Online]. Available: <https://www.research-collection.ethz.ch/entities/publication/c64a4d8c-b27d-4b1e-bde1-f94763316ca5>

Appendix A

Github repository

The source code for the project, while given in appendix, is also available at this URL: <https://github.com/Skeide-dev/Arm-Coresight-Frame-Deformatter-v2>.

The source code for the CSAL repository used: https://gitlab.inf.ethz.ch/project-openenzian/applications/etm-support/coresight-access-library/-/tree/master?ref_type=heads.

Appendix B

Block diagrams

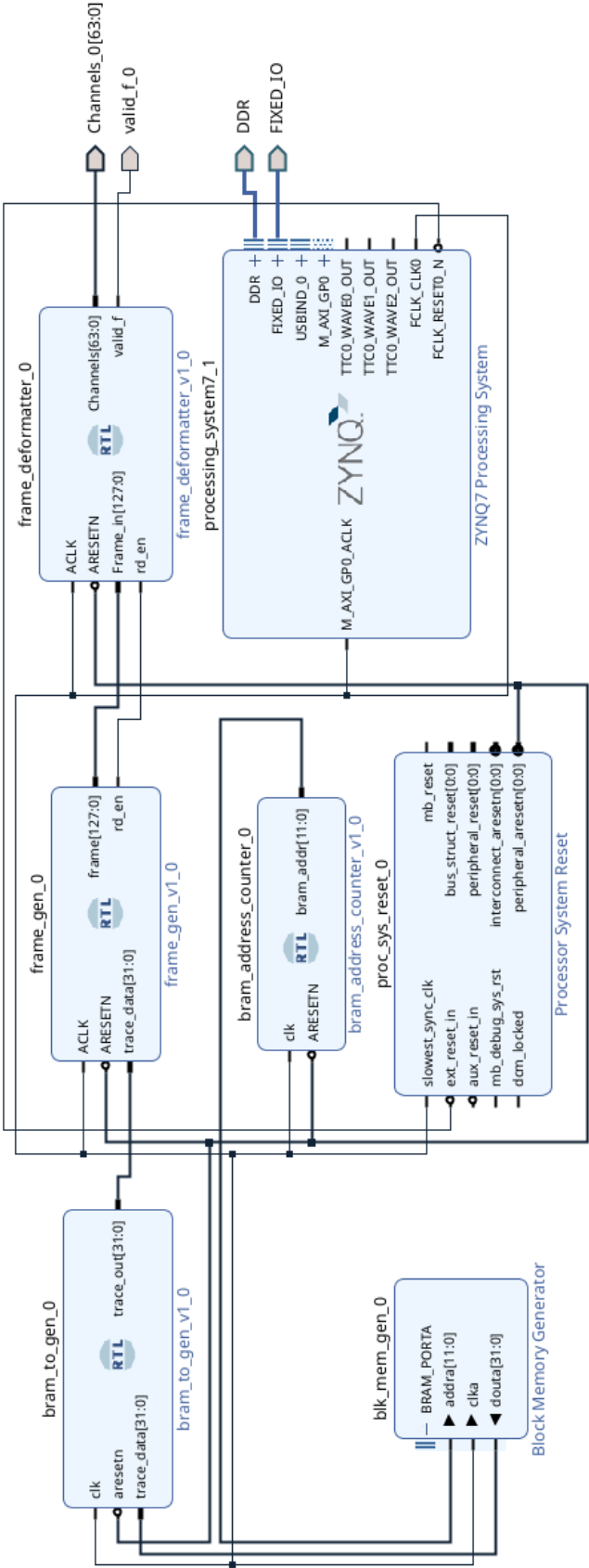


Figure B.1: BRAM infrastructure block diagram

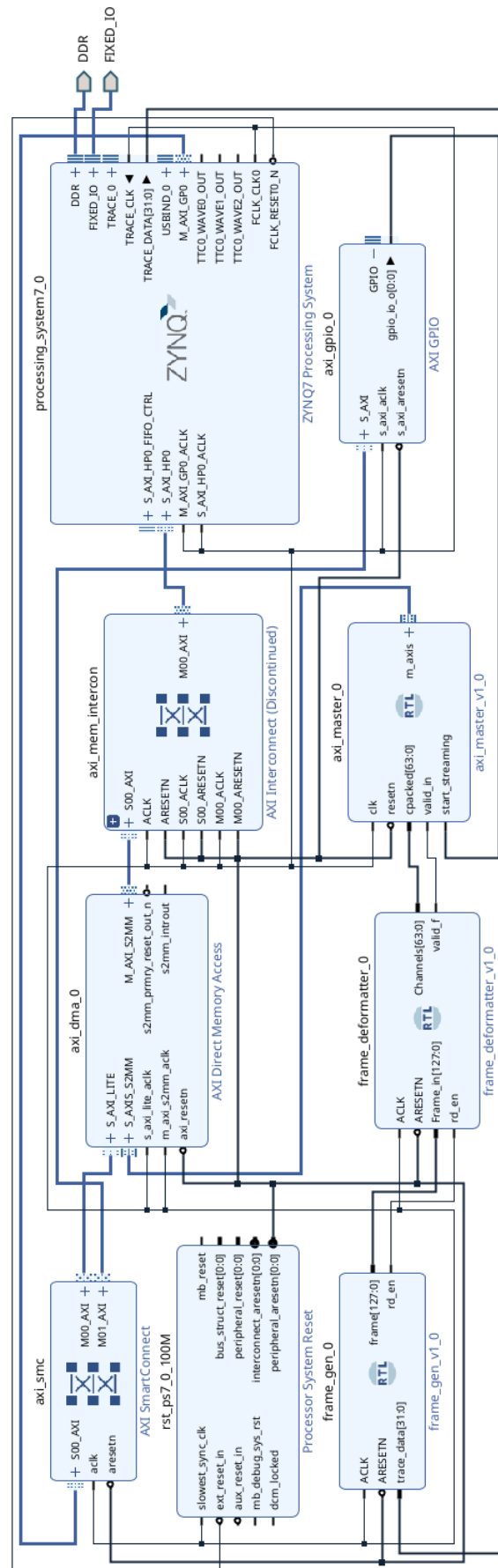


Figure B.2: TPIU block diagram

B.1 BRAM to Frame Generator

This is used to "fix" the delay described in Section 5.3.2, it is only used in the block diagram shown in Figure B.1.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity bram_to_gen is
5      Port (
6          clk          : in  STD_LOGIC;
7          aresetn       : in  STD_LOGIC;
8          trace_data    : in  STD_LOGIC_VECTOR(31 downto 0);
9          trace_out     : out STD_LOGIC_VECTOR(31 downto 0)
10     );
11 end bram_to_gen;
12
13 architecture Behavioral of bram_to_gen is
14
15     type t_state is (IDLE, ACTIVE);
16     signal state : t_state := IDLE;
17
18 begin
19
20     process(clk)
21     begin
22         if rising_edge(clk) then
23             if aresetn = '0' then
24                 state <= IDLE;
25             else
26                 case state is
27
28                     when IDLE =>
29                         trace_out <= x"7FFFFFFF";
30                         state      <= ACTIVE;
31
32                     when ACTIVE =>
33                         trace_out <= trace_data;
34
35                     end case;
36                 end if;
37             end if;
38         end process;
39

```

```
40 end Behavioral;
```

B.2 bram address counter

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity bram_address_counter is
6      Port (
7          clk          : in std_logic;
8          ARESETN      : in std_logic;
9          bram_addr    : out std_logic_vector(11 downto 0)
10     );
11 end bram_address_counter;
12
13 architecture Behavioral of bram_address_counter is
14     signal addr : unsigned(11 downto 0) := (others => '0');
15 begin
16     process(clk)
17     begin
18         if rising_edge(clk) then
19             if ARESETN = '0' then
20                 addr <= (others => '0');
21             else
22                 addr <= addr + 1;
23             end if;
24         end if;
25     end process;
26
27     bram_addr <= std_logic_vector(addr);
28 end Behavioral;
```

B.3 Frame Generator

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity frame_gen is
6      Port (
7          ACLK          : in STD_LOGIC;
```

```

8      ARESETN      : in  STD_LOGIC;
9      trace_data   : in  STD_LOGIC_VECTOR(31 downto 0);
10     -- Output to frame_decoder
11     frame        : out STD_LOGIC_VECTOR(127 downto 0);
12     rd_en        : out STD_LOGIC
13 );
14 end frame_gen;
15
16 architecture Behavioral of frame_gen is
17     signal frame_reg : std_logic_vector(127 downto 0);
18     constant SYNC_PATTERN : std_logic_vector(31 downto 0) := X"7FFFFFFF";
19     constant HALFWORD      : std_logic_vector(31 downto 0) := X"7FFF7FFF";
20     signal count           : unsigned(1 downto 0)           := "00";
21     signal error           : STD_LOGIC                     := '0';
22     signal begin_latch     : STD_LOGIC                     := '0';
23 begin
24
25
26
27     frame <= frame_reg;
28     process(ACLK)
29     begin
30         if rising_edge(ACLK) then
31             if ARESETN = '0' then
32                 rd_en <= '0';
33                 count <= "00";
34                 error <= '0';
35                 begin_latch <= '0';
36             else
37                 rd_en <= '0';
38
39                 if trace_data /= SYNC_PATTERN then -- Check for Full Frame Synch
40                     ↪ Packet
41                     if trace_data /= HALFWORD then -- Check for halfwords
42                         ↪ (idle-patterns)
43                     if begin_latch = '1' then -- before tpiu is configured it
44                         ↪ sends "0x00000001" words, the latch ignores them until the
45                         ↪ first sync frame is detected meaning the tpiu is turned
46                         ↪ on.
47                     -- Shift existing data left and add the new word at the
48                         ↪ bottom
49                     frame_reg <= trace_data & frame_reg(127 downto 32);
50
51                     if count = "11" then

```

```

46         -- On the 4th word, a full frame has been gathered,
         ↳ push read signal
47         rd_en <= '1';
48         count      <= "00";
49     else
50         count      <= count + 1; -- circular counter to check
         ↳ which 32-bit word we are at
51     end if;
52 end if; -- end latch check
53 end if; -- end halfword check
54 else
55 begin_latch <= '1'; -- first sync frame detected, TPIU is turned
         ↳ on.
56     if count /= "00" then
57         error <= '1'; -- synch packet appear inside frame, error!
58     else
59         error <= '0'; -- synchronized, reset the error flag!
60     end if;
61 end if; -- end frame synch
62 end if; -- end reset
63 end if; -- end clock
64 end process;
65 end Behavioral;

```

B.4 Frame Deformatter

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  package frame_types is
5      type channel_t is record
6          data : std_logic_vector(7 downto 0);
7          id   : std_logic_vector(6 downto 0);
8      end record;
9
10     type channel_array is array (0 to 3) of channel_t;
11     constant NULL_CHAN : channel_t := (data => (others => '0'), id => (others =>
         ↳ '0'));
12
13     -- The block diagram in vivado does not accept "records" as output/input so we
         ↳ map it over to vector
14     function to_vector(c : channel_t) return std_logic_vector;
15 end package;

```

```

16
17 package body frame_types is
18     function to_vector(c : channel_t) return std_logic_vector is
19     begin
20         -- Pack record into a standard 16-bit vector: [Data(8) | 0 | ID(7)]
21         return c.data & '0' & c.id;
22     end function;
23 end package body;
24
25 library IEEE;
26 use IEEE.STD_LOGIC_1164.ALL;
27 use work.frame_types.all;
28
29 entity frame_deformatter is
30     Port (
31         ACLK      : in  STD_LOGIC;
32         ARESETN    : in  STD_LOGIC;
33         Frame_in   : in  STD_LOGIC_VECTOR (127 downto 0);
34         rd_en      : in  STD_LOGIC;
35         -- Every clock cycle, we output 4 bytes of data, 4 bytes ID.
36         -- One valid signal so Axi_Master know when to store data into the buffer.
37         Channels    : out STD_LOGIC_VECTOR (63 downto 0);
38         valid_f     : out STD_LOGIC
39     );
40 end frame_deformatter;
41
42 architecture Behavioral of frame_deformatter is
43     type state_type is (IDLE, CYCLE_A, CYCLE_B, CYCLE_C);
44     signal state : state_type := IDLE;
45
46     signal new_id          : std_logic_vector(6 downto 0) := (others => '0');
47     signal old_id          : std_logic_vector(6 downto 0) := (others => '0');
48     signal NewID_isDelayed : std_logic := '0';
49     signal error           : std_logic := '0';
50     signal valid_output    : std_logic := '0';
51
52     type byte_array is array (0 to 14) of std_logic_vector(7 downto 0);
53     signal bytes_w, bytes_r : byte_array;
54     signal auxiliary_w, auxiliary_r : std_logic_vector(7 downto 0);
55
56     signal chan_regs : channel_array := (others => NULL_CHAN);
57
58 begin
59     -- Map internal records to the output ports using the conversion function

```

```

60 Channels <= to_vector(chan_regs(3)) & to_vector(chan_regs(2)) &
    ↪ to_vector(chan_regs(1)) & to_vector(chan_regs(0));

61
62 -- Wiring
63 auxiliary_w <= Frame_in(127 downto 120);
64 gen_mapping: for i in 0 to 14 generate
65     bytes_w(i) <= Frame_in((i*8)+7 downto i*8);
66 end generate;
67
68 process(ACLK)
69     variable v_selector : std_logic_vector(1 downto 0);
70 begin
71     if rising_edge(ACLK) then
72         if ARESETN = '0' then
73             state <= IDLE;
74             chan_regs <= (others => NULL_CHAN);
75             new_id <= (others => '0');
76             old_id <= (others => '0');
77             NewID_isDelayed <= '0';
78             error <= '0';
79         else
80             case state is
81                 when IDLE =>
82                     valid_output <= '0';
83                     if rd_en = '1' then
84                         valid_output <= '1';
85                         -- create a snapshot of the data into registers
86                         bytes_r <= bytes_w;
87                         auxiliary_r <= auxiliary_w;
88
89                         -- Clear channel 3 immediately (only 3 bytes this
90                         ↪ cycle)
91                         chan_regs(3) <= NULL_CHAN;
92                         -- Uses a variable here because vivado had issues
93                         ↪ accepting the case statements
94                         v_selector := bytes_w(2)(0) & bytes_w(0)(0);
95                         case v_selector is -- case statement for ID/DATA
96                             ↪ bytes, "1" if they are ID, "0" if they are Data
97                             when "00" =>
98                                 if NewID_isDelayed = '0' then -- was the
99                                     ↪ previous ID delayed?
100                                     chan_regs(0) <= (data => (bytes_w(0)(7
101                                         ↪ downto 1) & auxiliary_w(0)), id =>
102                                         ↪ new_id);

```

```

97         else -- Previous ID is delayed!
98             chan_regs(0) <= (data => (bytes_w(0)(7
               ↳ downto 1) & auxiliary_w(0)), id =>
               ↳ old_id);
99             NewID_isDelayed <= '0'; -- an ID can only
               ↳ be delayed for one data byte!
100         end if;
101         chan_regs(1) <= (data => bytes_w(1), id =>
               ↳ new_id);
102         chan_regs(2) <= (data => (bytes_w(2)(7 downto
               ↳ 1) & auxiliary_w(1)), id => new_id);
103
104
105     when "01" =>
106         chan_regs(0) <= NULL_CHAN; -- byte 0 is ID
107
108         if auxiliary_w(0) = '0' then -- is Byte 0 ID
               ↳ Delayed ?
109             chan_regs(1) <= (data => bytes_w(1), id =>
               ↳ bytes_w(0)(7 downto 1));
110         else -- Byte 0 ID is delayed! Use the previous
               ↳ ID for Byte 1!
111             chan_regs(1) <= (data => bytes_w(1), id =>
               ↳ new_id);
112         end if;
113         chan_regs(2) <= (data => (bytes_w(2)(7 downto
               ↳ 1) & auxiliary_w(1)), id => bytes_w(0)(7
               ↳ downto 1));
114
115         -- Save snapshots of ID's so next cycle know
               ↳ what to do.
116         old_id <= new_id;
117         new_id <= bytes_w(0)(7 downto 1);
118         NewID_isDelayed <= '0'; -- since we already
               ↳ have gone past the id's next byte, this
               ↳ must be 0.
119
120
121     when "10" =>
122         chan_regs(2) <= NULL_CHAN; -- Byte 2 is ID.
123
124         if NewID_isDelayed = '0' then -- was the
               ↳ previous ID delayed?

```

```

125         chan_regs(0) <= (data => (bytes_w(0)(7
            ↪  downto 1) & auxiliary_w(0)), id =>
            ↪  new_id);
126     else -- previous ID is delayed!
127         chan_regs(0) <= (data => (bytes_w(0)(7
            ↪  downto 1) & auxiliary_w(0)), id =>
            ↪  old_id);
128     end if;
129     chan_regs(1) <= (data => bytes_w(1), id =>
        ↪  new_id);
130
131     -- save ID snapshot for next cycle
132     old_id <= new_id;
133     new_id <= bytes_w(2)(7 downto 1);
134     NewID_isDelayed <= auxiliary_w(1); -- aux bits
        ↪  tell if ID's are delayed or not!
135
136
137     when "11" =>
138         chan_regs(0) <= NULL_CHAN; -- byte 0 is ID
139         chan_regs(2) <= NULL_CHAN; -- byte 2 is ID
140
141         if auxiliary_w(0) = '0' then -- is byte 0 ID
            ↪  delayed?
142             chan_regs(1) <= (data => bytes_w(1), id =>
                ↪  bytes_w(0)(7 downto 1));
143         else -- byte 0 ID is delayed!
144             chan_regs(1) <= (data => bytes_w(1), id =>
                ↪  new_id);
145         end if;
146
147         -- save ID snapshot for next cycle
148         old_id <= bytes_w(0)(7 downto 1);
149         new_id <= bytes_w(2)(7 downto 1);
150         NewID_isDelayed <= auxiliary_w(1); -- aux(1)
            ↪  tells next cycle if byte 2 is delayed or
            ↪  not!
151
152         when others => error <= '1'; -- Handles "U" "X"
            ↪  "Z" values, undefined values means something
            ↪  bad happened!
153     end case;

```

```

154         state <= CYCLE_A; -- Note: in the next cycles we will
        ↪ always decode 4 bytes instead of 3. This should help
        ↪ split the amount of logic gates across each cycle,
        ↪ since in IDLE state we also check for rd_en.
155     end if;
156
157
158     when CYCLE_A => -- Bytes 3, 4, 5, 6
159     valid_output <= '1';
160         v_selector := bytes_r(6)(0) & bytes_r(4)(0);
161         case v_selector is
162         when "00" =>
163             if NewID_isDelayed = '0' then -- Is the new_ID
        ↪ (byte 2) delayed ?
164                 chan_regs(0) <= (data => bytes_r(3), id =>
        ↪ new_id);
165             else
166                 chan_regs(0) <= (data => bytes_r(3), id =>
        ↪ old_id);
167                 NewID_isDelayed <= '0';
168             end if;
169             chan_regs(1) <= (data => (bytes_r(4)(7 downto
        ↪ 1) & auxiliary_r(2)), id => new_id);
170             chan_regs(2) <= (data => bytes_r(5), id =>
        ↪ new_id);
171             chan_regs(3) <= (data => (bytes_r(6)(7 downto
        ↪ 1) & auxiliary_r(3)), id => new_id);
172
173
174         when "01" =>
175             chan_regs(1) <= NULL_CHAN; -- byte4 is ID
176
177             if NewID_isDelayed = '0' then -- is new_ID
        ↪ (byte 2) delayed?
178                 chan_regs(0) <= (data => bytes_r(3), id =>
        ↪ new_id);
179             else -- new_ID (byte 2) is delayed!
180                 chan_regs(0) <= (data => bytes_r(3), id =>
        ↪ old_id);
181             end if;
182             if auxiliary_r(2) = '0' then -- is byte 4 ID
        ↪ delayed?
183                 chan_regs(2) <= (data => bytes_r(5), id =>
        ↪ bytes_r(4)(7 downto 1));

```

```

184         else -- byte 4 ID is delayed!
185             chan_regs(2) <= (data => bytes_r(5), id =>
186                 ↪ new_id);
187         end if;
188         chan_regs(3) <= (data => bytes_r(6)(7 downto
189             ↪ 1) & auxiliary_r(3), id => bytes_r(4)(7
190             ↪ downto 1));
191
192         old_id <= new_id;
193         new_id <= bytes_r(4)(7 downto 1);
194         NewID_isDelayed <= '0'; -- since we already
195             ↪ have gone past the id's next byte, this
196             ↪ must be 0.
197
198     when "10" =>
199         chan_regs(3) <= NULL_CHAN; -- byte 6 is an id
200
201         if NewID_isDelayed = '0' then -- is new_ID
202             ↪ (byte 2) delayed?
203             chan_regs(0) <= (data => bytes_r(3), id =>
204                 ↪ new_id);
205         else -- new_ID (byte 2) is delayed!
206             chan_regs(0) <= (data => bytes_r(3), id =>
207                 ↪ old_id);
208         end if;
209         chan_regs(1) <= (data => bytes_r(4)(7 downto
210             ↪ 1) & auxiliary_r(2), id => new_id);
211         chan_regs(2) <= (data => bytes_r(5), id =>
212             ↪ new_id);
213
214         old_id <= new_id;
215         new_id <= bytes_r(6)(7 downto 1);
216         NewID_isDelayed <= auxiliary_r(3);
217     when "11" =>
218         chan_regs(1) <= NULL_CHAN; -- byte 4 is ID
219         chan_regs(3) <= NULL_CHAN; -- byte 6 is ID
220
221         if NewID_isDelayed = '0' then -- Is new_ID
222             ↪ delayed?
223             chan_regs(0) <= (data => bytes_r(3), id =>
224                 ↪ new_id);
225         else -- new_ID is delayed!
226             chan_regs(0) <= (data => bytes_r(3), id =>
227                 ↪ old_id);

```

```

215         end if;
216         if auxiliary_r(2) = '0' then -- is byte 4 id
217             ↪ delayed?
218             chan_regs(2) <= (data => bytes_r(5), id =>
219                 ↪ bytes_r(4)(7 downto 1));
220         else -- byte 4 ID is delayed!
221             chan_regs(2) <= (data => bytes_r(5), id =>
222                 ↪ new_id);
223         end if;
224
225         old_id <= bytes_r(4)(7 downto 1);
226         new_id <= bytes_r(6)(7 downto 1);
227         NewID_isDelayed <= auxiliary_r(3);
228
229         when others =>
230             error <= '1';
231         end case;
232     state <= CYCLE_B;
233
234 when CYCLE_B => -- Bytes 7, 8, 9, 10
235     valid_output <= '1';
236     v_selector := bytes_r(10)(0) & bytes_r(8)(0);
237     case v_selector is
238     when "00" =>
239         if NewID_isDelayed = '0' then -- Is the new_ID
240             ↪ (byte 6) delayed ?
241             chan_regs(0) <= (data => bytes_r(7), id =>
242                 ↪ new_id);
243         else
244             chan_regs(0) <= (data => bytes_r(7), id =>
245                 ↪ old_id);
246             NewID_isDelayed <= '0';
247         end if;
248         chan_regs(1) <= (data => (bytes_r(8)(7 downto
249             ↪ 1) & auxiliary_r(4)), id => new_id);
250         chan_regs(2) <= (data => bytes_r(9), id =>
251             ↪ new_id);
252         chan_regs(3) <= (data => (bytes_r(10)(7 downto
253             ↪ 1) & auxiliary_r(5)), id => new_id);
254
255     when "01" =>
256         chan_regs(1) <= NULL_CHAN; -- byte 8 is ID

```

```

249     if NewID_isDelayed = '0' then -- is new_ID
        ↪ (byte 6) delayed?
250         chan_regs(0) <= (data => bytes_r(7), id =>
            ↪ new_id);
251     else -- new_ID (byte 6) is delayed!
252         chan_regs(0) <= (data => bytes_r(7), id =>
            ↪ old_id);
253     end if;
254     if auxiliary_r(4) = '0' then -- is byte 8 ID
        ↪ immediate?
255         chan_regs(2) <= (data => bytes_r(9), id =>
            ↪ bytes_r(8)(7 downto 1));
256     else -- byte 4 ID is delayed!
257         chan_regs(2) <= (data => bytes_r(9), id =>
            ↪ new_id);
258     end if;
259     chan_regs(3) <= (data => bytes_r(10)(7 downto
        ↪ 1) & auxiliary_r(5), id => bytes_r(8)(7
        ↪ downto 1));

260
261     old_id <= new_id;
262     new_id <= bytes_r(8)(7 downto 1);
263     NewID_isDelayed <= '0'; -- since we already
        ↪ have gone past the id's next byte, this
        ↪ must be 0.

264
265 when "10" =>
266     chan_regs(3) <= NULL_CHAN; -- byte 10 is an id

267
268     if NewID_isDelayed = '0' then -- is new_ID
        ↪ (byte 6) delayed?
269         chan_regs(0) <= (data => bytes_r(7), id =>
            ↪ new_id);
270     else -- new_ID (byte 6) is delayed!
271         chan_regs(0) <= (data => bytes_r(7), id =>
            ↪ old_id);
272     end if;
273     chan_regs(1) <= (data => bytes_r(8)(7 downto
        ↪ 1) & auxiliary_r(4), id => new_id);
274     chan_regs(2) <= (data => bytes_r(9), id =>
        ↪ new_id);

275
276     old_id <= new_id;
277     new_id <= bytes_r(10)(7 downto 1);

```

```

278         NewID_isDelayed <= auxiliary_r(5);
279     when "11" =>
280         chan_regs(1) <= NULL_CHAN; -- byte 8 is ID
281         chan_regs(3) <= NULL_CHAN; -- byte 10 is ID
282
283         if NewID_isDelayed = '0' then -- Is new_ID
284             ↪ delayed?
285             chan_regs(0) <= (data => bytes_r(7), id =>
286                 ↪ new_id);
287         else -- new_ID is delayed!
288             chan_regs(0) <= (data => bytes_r(7), id =>
289                 ↪ old_id);
290         end if;
291         if auxiliary_r(4) = '0' then -- is byte 8 id
292             ↪ delayed?
293             chan_regs(2) <= (data => bytes_r(9), id =>
294                 ↪ bytes_r(8)(7 downto 1));
295         else -- byte 8 ID is delayed!
296             chan_regs(2) <= (data => bytes_r(9), id =>
297                 ↪ new_id);
298         end if;
299
300         old_id <= bytes_r(8)(7 downto 1);
301         new_id <= bytes_r(10)(7 downto 1);
302         NewID_isDelayed <= auxiliary_r(5);
303     when others =>
304         error <= '1';
305     end case;
306     state <= CYCLE_C;
307
308 when CYCLE_C => -- Bytes 11, 12, 13, 14
309     valid_output <= '1';
310     v_selector := bytes_r(14)(0) & bytes_r(12)(0);
311     case v_selector is
312     when "00" =>
313         if NewID_isDelayed = '0' then -- Is the new_ID
314             ↪ (byte 10) delayed ?
315             chan_regs(0) <= (data => bytes_r(11), id
316                 ↪ => new_id);
317         else
318             chan_regs(0) <= (data => bytes_r(11), id
319                 ↪ => old_id);
320             NewID_isDelayed <= '0';
321         end if;

```

```

313     chan_regs(1) <= (data => (bytes_r(12)(7 downto
    ↪ 1) & auxiliary_r(6)), id => new_id);
314     chan_regs(2) <= (data => bytes_r(13), id =>
    ↪ new_id);
315     chan_regs(3) <= (data => (bytes_r(14)(7 downto
    ↪ 1) & auxiliary_r(7)), id => new_id);
316
317
318     when "01" =>
319         chan_regs(1) <= NULL_CHAN; -- byte 12 is ID
320
321         if NewID_isDelayed = '0' then -- is new_ID
    ↪ (byte 10) delayed?
322             chan_regs(0) <= (data => bytes_r(11), id
    ↪ => new_id);
323         else -- new_ID (byte 6) is delayed!
324             chan_regs(0) <= (data => bytes_r(11), id
    ↪ => old_id);
325         end if;
326         if auxiliary_r(6) = '0' then -- is byte 12 ID
    ↪ delayed?
327             chan_regs(2) <= (data => bytes_r(13), id
    ↪ => bytes_r(12)(7 downto 1));
328         else -- byte 12 ID is delayed!
329             chan_regs(2) <= (data => bytes_r(13), id
    ↪ => new_id);
330         end if;
331         chan_regs(3) <= (data => bytes_r(14)(7 downto
    ↪ 1) & auxiliary_r(7), id => bytes_r(12)(7
    ↪ downto 1));
332
333         old_id <= new_id;
334         new_id <= bytes_r(12)(7 downto 1);
335         NewID_isDelayed <= '0'; -- since we already
    ↪ have gone past the id's next byte, this
    ↪ must be 0.
336
337     when "10" =>
338         chan_regs(3) <= NULL_CHAN; -- byte 10 is an id
339
340         if NewID_isDelayed = '0' then -- is new_ID
    ↪ (byte 10) delayed?
341             chan_regs(0) <= (data => bytes_r(11), id
    ↪ => new_id);

```

```

342         else -- new_ID (byte 10) is delayed!
343             chan_regs(0) <= (data => bytes_r(11), id
344                 ↪ => old_id);
345         end if;
346         chan_regs(1) <= (data => bytes_r(12)(7 downto
347             ↪ 1) & auxiliary_r(6), id => new_id);
348         chan_regs(2) <= (data => bytes_r(13), id =>
349             ↪ new_id);
350
351         old_id <= new_id;
352         new_id <= bytes_r(14)(7 downto 1);
353         NewID_isDelayed <= auxiliary_r(7);
354
355     when "11" =>
356         chan_regs(1) <= NULL_CHAN; -- byte 12 is ID
357         chan_regs(3) <= NULL_CHAN; -- byte 14 is ID
358
359         if NewID_isDelayed = '0' then -- Is new_ID
360             ↪ delayed?
361             chan_regs(0) <= (data => bytes_r(11), id
362                 ↪ => new_id);
363         else -- new_ID is delayed!
364             chan_regs(0) <= (data => bytes_r(11), id
365                 ↪ => old_id);
366         end if;
367         if auxiliary_r(6) = '0' then -- is byte 12 id
368             ↪ delayed?
369             chan_regs(2) <= (data => bytes_r(13), id
370                 ↪ => bytes_r(12)(7 downto 1));
371         else -- byte 12 ID is delayed!
372             chan_regs(2) <= (data => bytes_r(13), id
373                 ↪ => new_id);
374         end if;
375
376         old_id <= bytes_r(12)(7 downto 1);
377         new_id <= bytes_r(14)(7 downto 1);
378         NewID_isDelayed <= auxiliary_r(7);
379
380     when others =>
381         error <= '1';
382     end case;
383
384     state <= IDLE;
385
386     when others => state <= IDLE;
387 end case;
388 end if;

```

```

377         end if;
378     end process;
379     valid_f <= valid_output;
380 end Behavioral;
381
382

```

B.5 AXI Master

A buffer which store the deformatted trace data and then stream it using the AXI Protocol to the AXI DMA.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity axi_master is
6      port (
7          clk          : in  std_logic;
8          resetn       : in  std_logic;
9          -- Input channels from frame_decoder
10         cpacked       : in  std_logic_vector(63 downto 0);
11         valid_in      : in  std_logic;
12         -- Control
13         start_streaming : in  std_logic; -- from GPIO: stop filling, start
           ↳ streaming
14         -- AXI4-Stream Master (to DMA S2MM)
15         m_axis_tdata   : out std_logic_vector(63 downto 0);
16         m_axis_tvalid  : out std_logic;
17         m_axis_tready  : in  std_logic;
18         m_axis_tkeep   : out std_logic_vector(7 downto 0);
19         m_axis_tlast   : out std_logic
20     );
21 end entity;
22
23 architecture rtl of axi_master is
24
25     type abuffer_t is array (0 to 1023) of std_logic_vector(63 downto 0);
26     signal abuffer : abuffer_t := (others => (others => '0')); -- initialized to
           ↳ zero, no padding needed
27
28     type state_t is (FILLING, STREAMING, DONE);
29     signal state : state_t := FILLING;

```



```

73         if rd_ptr = 1023 then
74             s_tlast  <= '1'; -- last data
75             state    <= DONE;
76         else
77             rd_ptr   <= rd_ptr + 1;
78         end if;
79     end if;
80
81
82     when DONE =>
83         if m_axis_tready = '1' then
84             s_tvalid <= '0';
85             s_tlast <= '0';
86         end if;
87         when others =>
88             state <= FILLING;
89
90     end case;
91 end if;
92 end if;
93 end process;
94 end architecture;

```

B.6 VITIS code

```

1  #include <string.h>
2  #include "xil_printf.h"
3  #include "ff.h"
4  #include "csaccess.h"
5  #include "csregisters.h"
6  #include "xil_mmu.h"
7  #include "xil_cache.h"
8  #include "sleep.h"
9  #include "xil_io.h"
10 #include "xaxidma.h"
11
12 #define ZYNQ_ROM_TABLE 0xF8800000
13 #define ZYNQ_PTMO      0xF889C000
14 #define ZYNQ_ETB       0xF8801000
15 #define ZYNQ_FUNNEL    0xF8804000
16 #define ZYNQ_TPIU      0xF8803000
17 #define PTMO_TRACE_ID  0x10
18

```

```

19 #define GPIO_BASE      0x41200000
20 #define DMA_DEVICE_ID    XPAR_AXI_DMA_0_BASEADDR // use device_id for vitis
    ↪ classic
21 #define DMA_TRANSFER_SIZE 1024                // Perform DMA-write of 1024 64-bit
    ↪ words
22
23 static XAxiDma dma_ctl;                        // AXI DMA driver instance
24 static XAxiDma_Config *dma_cfg;                // AXI DMA configuration parameters
25 int coresight_test1(int val);
26 static uint8_t gTraceBuffer[4096]; // ETB buffer
27 static int gTraceBufferLen;
28
29 static FATFS gFatFs;
30
31 static int CoreSightRegister(cs_device_t *pPTM0, cs_device_t *pETB, cs_device_t
    ↪ *pTPIU)
32 {
33     cs_device_t ptm0, etb, funnel, tpiu, rep;
34     int ret;
35
36     ret = cs_init();
37     if (ret)
38         return ret;
39
40     cs_diag_set(0);
41     cs_register_romtable(ZYNQ_ROM_TABLE);
42
43     ptm0 = cs_device_get(ZYNQ_PTM0);
44     etb = cs_device_get(ZYNQ_ETB);
45     funnel = cs_device_get(ZYNQ_FUNNEL);
46     tpiu = cs_device_get(ZYNQ_TPIU);
47     rep = cs_atb_add_replicator(2);
48
49     cs_device_set_affinity(ptm0, 0);
50     cs_atb_register(ptm0, 0, funnel, 0);
51     cs_atb_register(funnel, 0, rep, 0);
52     cs_atb_register(rep, 0, etb, 0);
53     cs_atb_register(rep, 1, tpiu, 0);
54     cs_registration_complete();
55
56     *pPTM0 = ptm0;
57     *pETB = etb;
58     *pTPIU = tpiu;
59

```

```
60     return ret;
61 }
62
63 static int CoreSightInit(cs_device_t *pPTM0, cs_device_t *pETB, cs_device_t
↪ *pTPIU)
64 {
65     cs_device_t ptm0, etb, tpiu;
66     cs_etm_config_t config;
67     int ret;
68
69     ret = CoreSightRegister(&ptm0, &etb, &tpiu);
70     if (ret)
71         return ret;
72
73
74
75     cs_disable_tpiu();
76     cs_sink_disable(etb);
77     cs_clear_trace_buffer(etb, 0);
78     cs_set_trace_source_id(ptm0, PTM0_TRACE_ID);
79     cs_etm_clean(ptm0);
80     cs_etm_config_init(&config);
81     config.flags = CS_ETMC_CONFIG;
82     cs_etm_config_get(ptm0, &config);
83     config.cr.raw.c.cxid_size = 1;
84     config.flags |= CS_ETMC_TRACE_ENABLE;
85     config.trace_enable_event = CS_ETME_WHEN(CS_ETMER_ALWAYS);
86     config.trace_enable_cr1 = CS_ETMTECR1_EXCLUDE;
87     config.trace_enable_cr2 = 0x00000000;
88     cs_etm_config_put(ptm0, &config);
89     cs_checkpoint();
90
91     *pPTM0 = ptm0;
92     *pETB = etb;
93     *pTPIU = tpiu;
94
95     return 0;
96 }
97
98 static int CoreSightBeginTrace(cs_device_t ptm0, cs_device_t etb, cs_device_t
↪ tpiu)
99 {
100
101     cs_sink_enable(etb);
```

```
102     cs_sink_enable(tpiu);
103     cs_trace_enable(ptm0);
104     cs_checkpoint();
105
106     return 0;
107 }
108
109
110
111 static int CoreSightStopTrace(cs_device_t ptm0, cs_device_t etb, cs_device_t tpiu)
112 {
113     int ret;
114     int unread_bytes;
115
116     ret = cs_trace_disable(ptm0);
117     cs_sink_disable(tpiu);
118     ret = cs_sink_disable(etb);
119
120     memset(gTraceBuffer, 0, sizeof(gTraceBuffer));
121     unread_bytes = cs_get_buffer_unread_bytes(etb);
122     ret = cs_get_trace_data(etb, gTraceBuffer, unread_bytes);
123     gTraceBufferLen = unread_bytes;
124     return ret;
125 }
126
127 static int MountSD(void)
128 {
129     FRESULT res = f_mount(&gFatFs, "0:", 1);
130     if (res != FR_OK) {
131         xil_printf("f_mount failed: %d\r\n", res);
132         return -1;
133     }
134     return 0;
135 }
136
137 static void UnmountSD(void)
138 {
139     f_mount(NULL, "0:", 0);
140 }
141
142 static int SaveTraceToBin(const char *filename, const uint8_t *buf, int len)
143 {
144     FIL      fil;
145     FRESULT  res;
```

```

146     UINT    bw;
147
148     Xil_DCacheFlush();
149
150     res = f_open(&fil, filename, FA_CREATE_ALWAYS | FA_WRITE);
151     if (res != FR_OK) {
152         xil_printf("f_open failed: %d\r\n", res);
153         return -1;
154     }
155
156     res = f_write(&fil, buf, (UINT)len, &bw);
157     if (res != FR_OK || (int)bw != len) {
158         xil_printf("f_write failed: res=%d written=%u\r\n", res, bw);
159         f_close(&fil);
160         return -1;
161     }
162
163     f_close(&fil);
164     return 0;
165 }
166
167 int main(void)
168 {
169     cs_device_t ptm0, etb, tpiu;
170     int ret;
171     int val;
172     u64 buf_acq_to_dma[DMA_TRANSFER_SIZE]; // DMA buffer for acquired data
173     s32 status;
174
175     // Mark CoreSight as strongly-ordered/device memory
176     // to prevent the CPU from reordering or caching MMIO accesses
177     Xil_SetTlbAttributes(0xF8800000, 0x10C06);
178     Xil_SetTlbAttributes(0x41200000, 0x10C06);
179     Xil_DCacheInvalidateRange((UINTPTR)buf_acq_to_dma, DMA_TRANSFER_SIZE*8);
180     xil_printf("=== ZedBoard ETB & TPIU tracing session ===\r\n");
181
182     ret = CoreSightInit(&ptm0, &etb, &tpiu);
183     if (ret) {
184         xil_printf("CoreSightInit failed: %d\r\n", ret);
185         return ret;
186     }
187
188     if (MountSD()) {
189         cs_shutdown();

```

```

190     return -1;
191 }
192
193 CoreSightBeginTrace(ptm0, etb, tpiu);
194
195 val = coresight_test1(20);
196 xil_printf("val: %d\r\n", val);
197
198 CoreSightStopTrace(ptm0, etb, tpiu);
199
200 xil_printf("ETB captured %d bytes of trace\r\n", gTraceBufferLen);
201
202 // Initialize AXI DMA driver
203 dma_cfg = XAxiDma_LookupConfig(DMA_DEVICE_ID);
204
205 if (NULL == dma_cfg) {
206     return XST_FAILURE;
207 }
208 status = XAxiDma_CfgInitialize(&dma_ctl, dma_cfg);
209 if (status != XST_SUCCESS) {
210     return XST_FAILURE;
211 }
212 // Disable DMA interrupt (use polling)
213 XAxiDma_IntrDisable(&dma_ctl, XAXIDMA_IRQ_ALL_MASK, XAXIDMA_DEVICE_TO_DMA);
214 /* Data acquisition and DMA */
215
216 // Enable AXI streaming data in Pl
217 Xil_Out32(GPIO_BASE, 0x1);
218
219 // Submit for DMA-write operation to move data to system memory
220 status = XAxiDma_SimpleTransfer(&dma_ctl, (UINTPTR)buf_acq_to_dma,
    ↪ DMA_TRANSFER_SIZE*8, XAXIDMA_DEVICE_TO_DMA);
221 if (status != XST_SUCCESS) {
222     return XST_FAILURE;
223 }
224
225 usleep(2);
226
227 while (XAxiDma_Busy(&dma_ctl, XAXIDMA_DEVICE_TO_DMA));
228
229
230
231 if (gTraceBufferLen <= 0) {
232     xil_printf("No ETB trace data captured\r\n");

```

```
233     UnmountSD();
234     cs_shutdown();
235     return 1;
236 }
237
238 ret = SaveTraceToBin("0:trace.bin", gTraceBuffer, gTraceBufferLen);
239 if (ret) {
240     xil_printf("Failed to save ETB trace to SD\r\n");
241     UnmountSD();
242     cs_shutdown();
243     return 1;
244 }
245
246 xil_printf("ETB trace saved to trace.bin\r\n");
247 xil_printf("First 5 frames of ETB trace:\r\n");
248 for (int i = 0; i < 80; i++) {
249     xil_printf("%02x ", gTraceBuffer[i]);
250     if ((i + 1) % 16 == 0)
251         xil_printf("\r\n");
252 }
253
254 ret = SaveTraceToBin("0:tpiu1.bin", buf_acq_to_dma, DMA_TRANSFER_SIZE*8);
255 if (ret) {
256     xil_printf("Failed to save DMA trace to SD\r\n");
257     UnmountSD();
258     cs_shutdown();
259     return 1;
260 }
261 xil_printf("DMA trace saved to tpiu1.bin\r\n");
262 xil_printf("First 5 frames of DMA trace:\r\n");
263 for (int i = 0; i < 80; i++) {
264     xil_printf("%02x ", buf_acq_to_dma[i]);
265     if ((i + 1) % 16 == 0)
266         xil_printf("\r\n");
267 }
268
269 UnmountSD();
270 cs_shutdown();
271 return 0;
272 }
```

B.7 C# Comparison Script

```

1  /* This program is designed to compare a file of decoded CoreSight trace in the
   ↪ openCSD format,
2  * and a file of decoded CoreSight trace from our deformatter that is extracted
   ↪ from simulation
3  */
4  using System;
5  namespace CS_Comparator
6  {
7      internal class Program
8      {
9          static void Main(string[] args)
10         {
11             StreamReader sr_BS = File.OpenText("fire_channels.csv");// opens file
12             StreamReader sr_OCSD = File.OpenText("t1_decoded.txt");// opens file
13
14             for (int i = 0; i < 6; i++)// removes the lines with no data
15             {
16                 sr_OCSD.ReadLine();
17             }
18             sr_BS.ReadLine();// removes the lines with no data
19             sr_BS.ReadLine();
20             // make variables for processing of lines in file
21             string bs_Line = "";
22             string[] Ar_BS= new string[0]; // Make empty, .split() will make array
23             string[] Ar_OCSD = new string[0];
24             int Ar_OCSD_i = 5;// data starts in the 6th element in OpenCSD file.
25             // initialize global variables for the IDs and the data
26             // they are global to save memory usage and make debugging easier
27             string BS_ID = "";
28             string OCSD_ID = "";
29             string BS_Data = "";
30             string OCSD_Data = "";
31             bool Done = false;
32             bool new_line_OCSD = true;
33             bool isDifferent = false;
34             while ( !Done)
35             {
36                 bs_Line = sr_BS.ReadLine();
37                 if (bs_Line == null)// Null only at the end of file
38                     break;
39                 Ar_BS = bs_Line.Split(' ', ',',
   ↪ StringSplitOptions.RemoveEmptyEntries); //

```

```

40     if (new_line_OCSD) // executes only the first while loop
41     {
42         ReadLineSrOCSD(sr_OCSD, ref Ar_OCSD, ref OCSD_ID); // calls
43         → method, this sets the array and the ID for the data in it
44         new_line_OCSD=false; // disables the if
45     }
46     for (int i = 1; i < 5; i++) // data is in element 1-4, element zero
47     → is sample time
48     {
49         if (Ar_BS[i].Substring(2, 2) == "00" || Ar_BS[i].Substring(2,
50         → 2) == "20") // Id 20 is not in OpenCSD file and the 00 id
51         → is invalid data
52         { continue; } //skip loop instance
53         // separates ID and data to compare later
54         BS_ID = Ar_BS[i].Substring(2, 2).ToLower(); // make lower case
55         → to match format of OpenCSD
56         BS_Data = Ar_BS[i].Substring(0, 2).ToLower(); // make lower
57         → case to match format of OpenCSD
58         OCSD_Data = Ar_OCSD[Ar_OCSD_i]; // find the data of the
59         → OpenCSD file
60         if (BS_ID == OCSD_ID && BS_Data == OCSD_Data) // comparing ID
61         → to ID and data to data
62         {
63             Ar_OCSD_i++; // increase the index to the next data
64             if (Ar_OCSD_i >= Ar_OCSD.Length) // check if the end of
65             → line is reached
66             {
67                 ReadLineSrOCSD(sr_OCSD, ref Ar_OCSD, ref
68                 → OCSD_ID); //update ID and array
69                 Ar_OCSD_i = 5; // reset to start of data
70             }
71         }
72         else
73         {
74             Console.Write($"Last OpenCSD index checked {Ar_OCSD[3]}
75             → "); // write last index the program got to
76             Console.WriteLine($"Last sample: {Ar_BS[0]} ");
77             Console.WriteLine("Mismatch detected.");
78             isDifferent = true;
79             Done = true;
80             break;
81         }
82     }
83 }
84 // check if the end of files is reached

```

```

73         if (sr_BS.EndOfStream && sr_OCSD.EndOfStream)
74             { Done = true; }
75     }
76     if (!isDifferent)
77     {
78         Console.WriteLine($"Last OpenCSD index checked {Ar_OCSD[3]} ");//
79         ↪ write indexes in OpenCSD file that have been checked
80         Console.WriteLine();
81         Console.WriteLine("No mismatches detected.");
82     }
83     // close files
84     sr_BS.Close();
85     sr_OCSD.Close();
86 }
87 static void ReadLineSrOCSD(StreamReader sr_OCSD, ref string[] Ar_OCSD, ref
88 ↪ string OCSD_ID)
89 {
90     string OCSD_Line =sr_OCSD.ReadLine();
91     if (OCSD_Line == null)// the end of file is reached, do nothing
92         return;
93     Ar_OCSD = OCSD_Line.Split(' ', ';',
94     ↪ StringSplitOptions.RemoveEmptyEntries); // making new array (the
95     ↪ length of the array is also changed (if it is different))
96     OCSD_ID = string.Concat(Ar_OCSD[4][10], Ar_OCSD[4][11]); // making new
97     ↪ ID, this is the same for the whole line
98 }
99 }
100 }

```

B.8 TCL script for simulation

This is the .tcl script used to extract data from simulation into a .csv file.

```

1
2 restart
3
4 set fp [open "/home/noctua1/project_1/fire_channels.csv" w]
5 puts $fp "Time_ns, Chan0, Chan1, Chan2, Chan3"
6
7
8 set step      8
9 set total     19000
10 set recording 0

```

```

11 set done          0
12 set prev_rd_en 0
13
14
15 for {set t 0} {$t <= $total && $done == 0} {incr t $step} {
16
17     set rd_en      [get_value
18         ↪ {/bramtrace_wrapper/bramtrace_i/frame_deformatter_0/rd_en}]
19
20     set bram_addr [get_value -radix hex
21         ↪ {/bramtrace_wrapper/bramtrace_i/bram_address_counter_0_bram_addr}]
22
23     # Read 64-bit Channels_0 as hex, then convert
24     set raw64 [get_value -radix hex {/bramtrace_wrapper/Channels_0}]
25     # Remove any leading 0x if present
26     set raw64 [string trim $raw64]
27     set raw64 [regsub {^0x} $raw64 ""]
28     # split into 4 x 16-bit fields
29     set hex3 [string range $raw64 0 3]
30     set hex2 [string range $raw64 4 7]
31     set hex1 [string range $raw64 8 11]
32     set hex0 [string range $raw64 12 15]
33
34     # Detect rising edge of rd_en
35     if {$recording == 0 && $prev_rd_en == 0 && $rd_en == 1} {
36         set recording 1
37         puts " rd_en HIGH at ${t}ns - recording started"
38     }
39
40     # Write data only while recording
41     if {$recording == 1} {
42         # Write row to CSV
43         puts $fp "$t, $hex0, $hex1, $hex2, $hex3"
44
45         # Stop when bram_addr reaches 908
46         if {$bram_addr == "908"} {
47             set done 1
48             puts " bram_addr reached 908 at ${t}ns - recording stopped"
49         }
50     }
51 }
52
53 set prev_rd_en $rd_en
54 run ${step}ns
55
56 }

```

```
53 close $fp
54 puts " CSV saved."
```

Appendix C

ILA simulation

Figure C.1 is the full ILA simulation, which show the trace data received from the TPIU, rd_en pulses high every time 4 words are gathered, and channels[63:0] is the deformatted output which is stored in the buffer in axi_master. Once GPIO is set to 1, axi_master generate the axi protocol to stream data to the dma, here we only have 400 bytes of trace data, the rest of the buffer is filled with zeroes.

Shown here as it's too large of a simulation sample to create a detailed diagram out of.

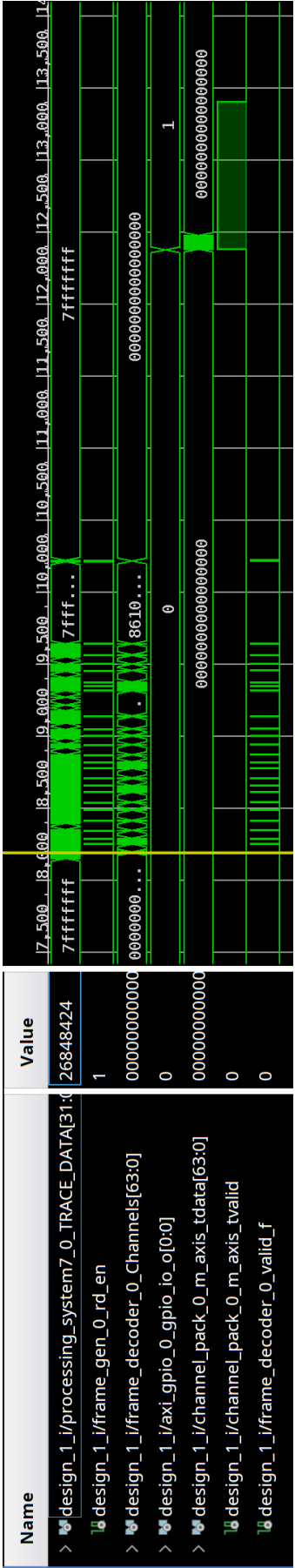


Figure C.1: Full ila simulation

Appendix D

Risk Analysis

Keyword	Description	Probability	Consequence	Preventive action
Technical / Hardware				
Hardware-fault	Defect zedboard	Low	High	Test zedboard with simple program before the coding and testing of the final product on the zedboard.
Timing fault in FPGA	The deformatter can't compute the 32-bit data at the wanted clock frequency.	Medium	Med-high	Pipelining, and execution of timing-analysis early in the project.
Synchronizing	Faults in detection of frame start (half-word alignment).	Medium	High	Implement dedicated sync-detector for "half-word"-patterns.
Data / Overflow				

Keyword	Description	Probability	Consequence	Preventive action
Internal Overflow	The stream of data from TPIU is greater than the AXI component's BRAM can handle.	Medium	High	Make the BRAM storage bigger than the ETB storage, since both will be used together and program execution is stopped when ETB is full.
External Overflow	CPU generates data quicker than TPIU-interface can send out for short periods.	High	High	Configure ETF (Embedded Trace FIFO) as an elastic buffer to absorb data-peaks from the processor.
Knowledge / Competence				
Competence gap	The protocol (CoreSight) is more complex than assumed.	Med-low	Medium	Thorough reading of documentation and other works on the subject.
Project Management				
Development time	Unforeseen problems might slow down development time.	High	Medium	Use efficient methods, make plans and communicate with the team.
Progress plan	Time estimates are too optimistic, with little room for unforeseen issues.	Med-High	Medium	Continuously follow up on milestones. With critical delay, prioritize core functionality above new features.
Team / Collaboration				

Keyword	Description	Probability	Consequence	Preventive action
Version conflict	Group members overwrite each other's VHDL code.	Medium	Low	Consistent use of Git/GitHub and tcl-script for Vivado project.
Resources / Access				
Lab-access	Limited access to lab resources.	Medium	Medium	Focus on development with the zedboard.

Table D.1: Risk Analysis

Appendix E

Design Requirements Analysis

E.1 Functional requirements

Nr.	Requirement	Description
F1	Test on data from memory	The system shall be able to be initialized and tested with predefined trace data stored in memory (for simple verification).
F2	Connect to the TPIU	The system should be able to connect to and decode data directly from the TPIU.
F3	Synchronize frame	The deformatter should identify the synchronization sequences in the CoreSight-stream, and discard these.
F4	Separate data stream	The deformatter should be able to differentiate between id and data bytes, and also separate data based on the different source-IDs.
F5	Store deformatted data	The system should store the processed data and make it accessible for further processing later.

Table E.1: Functional requirements for the system

E.2 Non-Functional requirements

Nr.	Requirement	Description
NF1	Modularity	The design should be composed of modules for easier testing, troubleshooting and possible future expansions.
NF2	Simulability	The entire pipeline must be thoroughly tested using simulation before implementation on hardware.
NF3	Resource awareness	Implementation should be efficient with usage off FPGA resources.
NF4	Documentation	The code and the system architecture should be documented to facilitate further development of the project, also by others.

Table E.2: Non-Functional requirements for the system

E.3 Progress plan

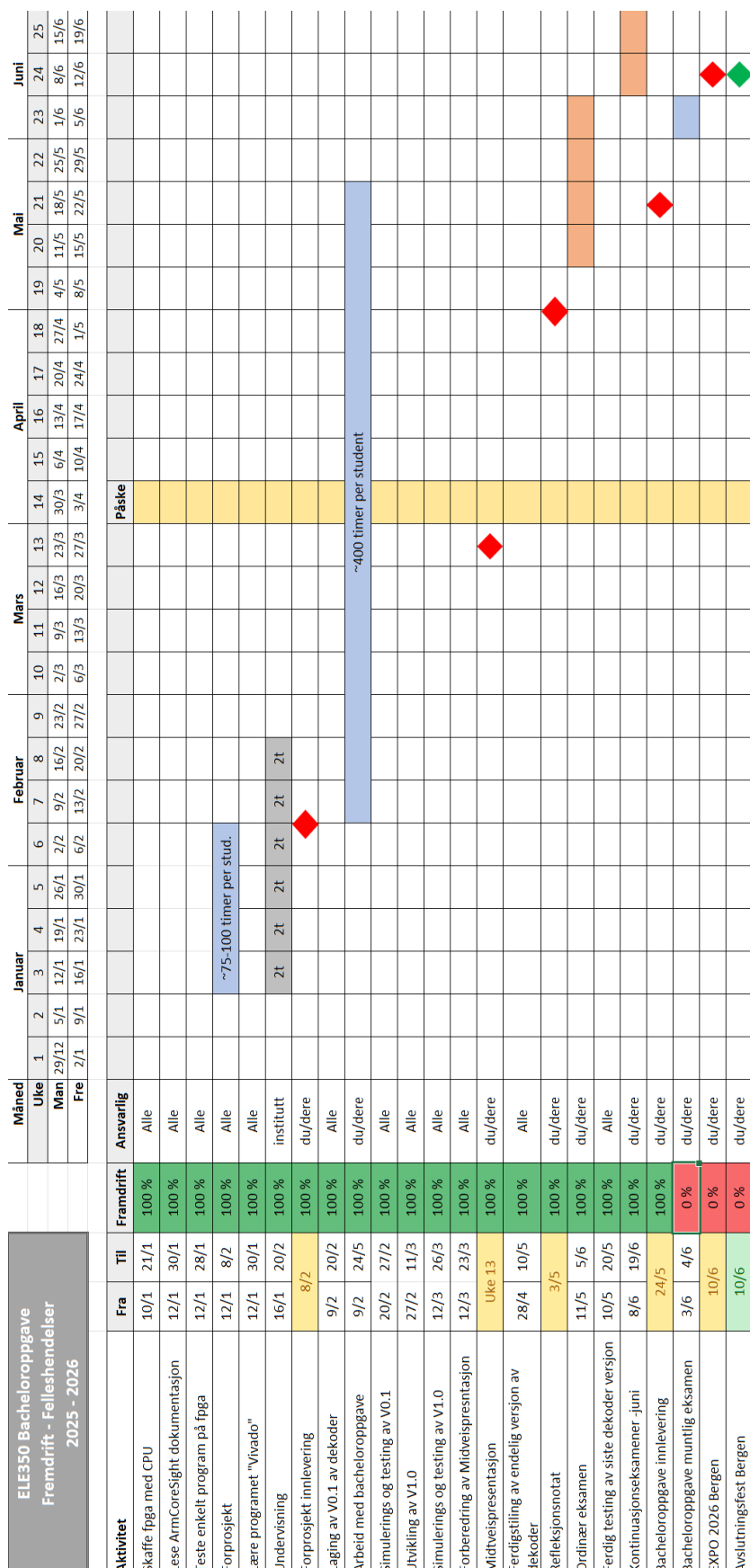


Figure E.1: Progress plan

Appendix F

Use of AI Tools

We used AI tools such as Microsoft Copilot and Claude as learning aids throughout the project, particularly for understanding the ARM CoreSight architecture, navigating unfamiliar tools such as Vivado and Vitis, setting up and debugging a Yocto-based Linux build, and working with Linux/Ubuntu commands for the first time. Beyond onboarding, the tools served as a sparring partner for iterative improvement of our code and verification scripts. We also used AI assistance to improve the flow and structure of written drafts in this report.

Time Sheet ING350

Group: BO26EB-43 - "Arm CoreSight Protocol Decoder" - ELN_06
Study: Electronics Engineering
Name: Georg Ansgar Skeide
Candidate: 325
Responsibility: Projectleader, Systemintegrator, Modelling
Description of responsibility: Responsible for overall planning, progress, and coordination of the group's resources. Technically responsible for modelling the ARM CoreSight architecture, developing high-level block diagrams, and ensuring that the interfaces between the VHDL modules and the rest of the system function correctly (system integration). Also handles VITIS (SW).

day	date	hours	short description
Sunday	18.01	3	Read through references in the assignment
Monday	19.01	4	Read through references in the assignment
Tuesday	20.01	4	Meeting with the group to organize attendance, tasks and tools
Tuesday	20.01	5	Worked on setting up a LaTeX template for the group
Wednesday	21.01	2	Meeting with main supervisor and client
Wednesday	21.01	5	Group learning the Vivado tool
Thursday	22.01	5	Group learning the Vivado tool
Thursday	22.01	2	Read through Zynq7000 TRM and debug architecture
Friday	23.01	3	Set up LaTeX pre-project template for the group
Saturday	24.01	2	Group agreed on the Overleaf setup
Saturday	24.01	4	Wrote in the pre-project
Monday	26.01	4	Read up on ARM CoreSight v3.0 TRM
Tuesday	26.01	7	Read up on ARM CoreSight v3.0 TRM
Wednesday	28.01	1	Meeting with client
Wednesday	28.01	4	Group reviewing ARM CoreSight theory
Thursday	29.01	1	Meeting with main supervisor
Thursday	29.01	5	Group learning the Vivado simulation tool
Friday	30.01	4	Read through TMC TRM and Zedboard TRM and wrote pre-project
Tuesday	03.02	4	Wrote some in the pre-project
Wednesday	04.02	5	Group working on the pre-project
Wednesday	04.02	3	Wrote in the pre-project

Thursday	05.02	1	Meeting with main supervisor about the supervision agreement
Thursday	05.02	5	Group working on the pre-project
Friday	06.02	3	Group working on the pre-project
Friday	06.02	5	Working on the pre-project
Saturday	07.02	7	Wrote in the pre-project
Sunday	08.02	9	Wrote in the pre-project
Wednesday	11.02	1	Meeting with client
Wednesday	11.02	5	Working with the group
Thursday	12.02	1	Meeting with main supervisor
Thursday	12.02	5	Working with the group
Tuesday	17.02	7	Set up some diagrams/figures for the report
Wednesday	18.02	5	Working with the group
Thursday	19.02	0.1	Meeting with main supervisor
Thursday	19.02	5	Working with the group
Friday	20.02	4	Reviewing trace blobs from client
Saturday	21.02	6	Reading through TRM documents to understand why the trace blobs are not as expected
Wednesday	25.02	3	Working with the group
Thursday	26.02	5	Working with the group
Saturday	28.02	6	Reading up on OpenCSD
Sunday	01.03	5	Working on decoding trace blobs via OpenCSD
Wednesday	04.03	4	Working with the group
Thursday	05.03	1	Meeting with client
Thursday	05.03	5	Working with the group
Friday	06.03	5	Working with the group
Friday	06.03	3	Reading up on Yocto Project
Saturday	07.03	7	Trying to get Linux running via Yocto Project
Sunday	08.03	8	Debugging the Linux system
Monday	09.03	4	Got Linux working for Zedboard, trying to include CoreSight components
Tuesday	10.03	6	Helping Saleh with simulation
Wednesday	11.03	4	Working with the group
Thursday	12.03	4	Group meeting and report writing
Thursday	12.03	5	Reading up on CSAL and cross-compiling it into the Linux build
Friday	13.03	8	Writing a C script to get trace data to ETB and save as binary file to SD card
Saturday	14.03	7	Switched the CSAL library to another one with TPIU support; log shows TPIU working but needs testing

Wednesday	18.03	4	Group meeting and discussion about further work
Thursday	19.03	4	Working with the group
Wednesday	23.03	2	Meeting to work on presentation
Thursday	24.03	3	Meeting to work on presentation
Wednesday	08.04	4	Group meeting
Thursday	09.04	3	Group meeting
Thursday	09.04	5	Working with AXI and CoreSight through VITIS
Friday	10.04	9	Working with AXI and CoreSight through VITIS
Monday	13.04	8	Working with AXI and writing in report
Tuesday	14.04	8	Drawing diagrams for report
Wednesday	15.04	4	Group meeting
Wednesday	15.04	6	Working with AXI
Thursday	16.04	4	Group meeting
Thursday	16.04	7	Debugging AXI
Friday	16.04	10	Debugging AXI
Monday	20.04	7	Working on report writing
Tuesday	21.04	6	Report writing
Wednesday	22.04	4	Group meeting
Wednesday	22.04	6	Working with AXI
Thursday	23.04	4	Group meeting
Thursday	23.04	8	Reading/working on ILA
Friday	24.04	8	Reading/working on ILA
Monday	27.04	7	Report writing
Tuesday	28.04	5	Report writing
Wednesday	29.04	4	Group meeting
Wednesday	29.04	7	Writing in report
Thursday	30.04	4	Group meeting
Thursday	30.04	8	Reading and working in report
Friday	01.05	8	Reading up on theory
Monday	04.05	4	Report writing
Tuesday	05.05	7	Worked with VITIS
Wednesday	06.05	4	Group meeting
Wednesday	06.05	7	Reading up on references
Thursday	07.05	1	Meeting with supervisor Endre
Thursday	07.05	3	Group meeting
Thursday	07.05	5	Writing in report
Friday	08.05	8	Writing in report
Saturday	09.05	5	Drawing diagrams for report
Monday	11.05	8	Reading up on references

Tuesday	12.05	10	Report writing
Wednesday	13.05	4	Group meeting
Wednesday	13.05	8	Fixing DMA
Thursday	14.05	4	Group meeting
Thursday	14.05	10	Writing in report
Friday	15.05	11	Writing in report
Saturday	16.05	10	Writing in report
Monday	18.05	2	Group meeting
sum		512 hours	

Time Sheet ING350

Group: BO26EB-43 - "Arm CoreSight Protocol Decoder" - ELN_06
Field of study: Electronics Engineering
Project participant name: Johan Romarheim
Candidate number: 174
Area of responsibility: Logic Designer (VHDL responsible)
Description of responsibility: Responsible for the design and coding of the core functionality in VHDL. This involves developing logic to decode CoreSight protocols, with a focus on ensuring the code is synthesizable and meets timing-closure requirements, as well as optimizing the code to satisfy the project's performance requirements.

day	date	hours	short description
Monday	19.01	7	Reading documents
Tuesday	20.01	4	Meeting with the group to organize attendance, tasks and tools
Wednesday	21.01	2	Meeting with main supervisor and client
Wednesday	21.01	5	Group learning the Vivado tool
Thursday	22.01	5	Group learning the Vivado tool
Friday	23.01	6	Work on deformatter
Saturday	24.01	2	Group agreed on the Overleaf setup
Monday	26.01	4	Reading CoreSight documentation
Tuesday	27.01	4	Reading master's thesis from Zürich
Wednesday	28.01	1	Meeting with client
Wednesday	28.01	4	Group reviewing ARM CoreSight theory
Thursday	29.01	1	Meeting with main supervisor
Thursday	29.01	5	Group learning the Vivado simulation tool
Friday	30.01	6	Work on pre-project and frame generator
Wednesday	04.02	5	Group working on the pre-project
Thursday	05.02	1	Meeting with main supervisor about the supervision agreement
Thursday	05.02	5	Group working on the pre-project
Friday	06.02	3	Group working on the pre-project
Saturday	07.02	5	Working on pre-project
Tuesday	10.02	4	Work on deformatter V0 and generator
Wednesday	11.02	1	Meeting with client
Wednesday	11.02	5	Working with the group

Thursday	12.02	1	Meeting with main supervisor
Thursday	12.02	5	Working with the group
Saturday	14.02	4	Writing report
Monday	16.02	4	Work on Deformatter V0
Tuesday	17.02	6	Work on Deformatter V0
Wednesday	18.02	5	Working with the group
Thursday	19.02	0.1	Meeting with main supervisor
Thursday	19.02	5	Working with the group
Friday	20.02	4	Work on deformatter V0
Saturday	21.02	4	Work on deformatter and explanatory figures
Wednesday	25.02	3	Working with the group
Thursday	26.02	5	Working with the group
Monday	02.03	3	Writing report
Tuesday	03.03	4	Work on Deformatter V1
Wednesday	04.03	4	Working with the group
Thursday	05.03	1	Meeting with client
Thursday	05.03	5	Working with the group
Friday	06.03	5	Writing on project
Monday	09.03	7	Work on Deformatter V1
Wednesday	11.03	4	Working with the group
Thursday	12.03	4	Group meeting and report writing
Friday	13.03	5	AXI DMA research
Saturday	14.03	4	Reviewing work from another group member
Monday	16.03	7	Work on AXI DMA
Wednesday	18.03	4	Group meeting and discussion about further work
Thursday	19.03	4	Working with the group
Friday	20.03	3	Work on midterm presentation and AXI DMA
Saturday	21.03	2	Work on AXI DMA
Monday	23.03	2	Meeting to work on presentation
Tuesday	24.03	5	Meeting to work on presentation
Wednesday	25.03	2	Preparation for presentation
Thursday	26.03	5	Work on AXI DMA
Friday	27.03	2	Work on AXI DMA
Saturday	28.03	10	Testing AXI DMA writing to SD card
Sunday	29.03	6	Debugging AXI write pipeline
Monday	30.03	6	Writing decoded file from FPGA to SD card
Tuesday	31.03	6	File formatting and comparison
Wednesday	01.04	6	Work on data comparison
Tuesday	07.04	6	Report writing

Wednesday	08.04	5	Working with the group
Thursday	09.04	7	Working with the group and report writing
Friday	10.04	9	Report writing
Saturday	11.04	3	Report writing
Monday	13.04	6	Report writing
Tuesday	14.04	8	Reading and English presentation
Wednesday	15.04	8	Group work and programming comparison script
Thursday	16.04	6	Working with the group
Friday	17.04	6	Testing comparison script
Saturday	18.04	7	Report writing
Monday	20.04	8	Report writing
Tuesday	21.04	7	Report writing
Wednesday	22.04	6	Report writing
Thursday	23.04	7	Work on report
Friday	24.04	6	Work on report
Monday	27.04	8	Work on report
Tuesday	28.04	8	Work on report
Wednesday	29.04	7	Group work and writing in report
Thursday	30.04	7	Report writing
Friday	01.05	9	Report writing
Monday	04.05	8	Report writing
Tuesday	05.05	8	Writing reflection note
Wednesday	06.05	8	Report writing
Thursday	07.05	8	Report writing
Friday	08.05	10	Report writing
Saturday	09.05	7	Report writing
Monday	11.05	8	Report writing
Tuesday	12.05	8	Report writing
Wednesday	13.05	6	Report writing
Thursday	14.05	7	Report writing
Friday	15.05	8	Report writing
Saturday	16.05	8	Report writing
Monday	18.05	8	Report writing
Tuesday	19.05	10	Report writing
sum		456 hours	

Time Sheet ING350

Group: BO26EB-43 - "Arm CoreSight Protocol Decoder" - ELN_06

Field of study: Electronics Engineering

Project participant name: Saleh Barakat

Candidate number: 190

Area of responsibility: Test and Verification Responsible

Description of responsibility: Responsible for quality assurance and functional validation of the system. This involves developing testbenches in VHDL, preparing test plans, and conducting simulations. The person holds primary responsibility for verifying that the logic and algorithm conform to the specifications, as well as documenting test results and handling debugging in collaboration with the developers.

day	date	hours	short description
Tuesday	20.01	4	Group meeting to organize the work of the assignment
Tuesday	20.01	5	Home work: began reading ARM theory
Wednesday	21.01	2	Meeting with main supervisor and client
Wednesday	21.01	5	Group learning the Vivado tool
Wednesday	21.01	5	Home work: Vivado tool, reading theory
Thursday	22.01	5	Group learning the Vivado tool
Thursday	22.01	5	Home work: Vivado tool and ARM theory
Saturday	24.01	2	Group agreed on the Overleaf setup
Saturday	24.01	5	Home work: Vivado tool, reading theory
Wednesday	28.01	1	Meeting with client
Wednesday	28.01	4	Group reviewing ARM CoreSight theory
Wednesday	28.01	5	Home work: reading ARM theory
Thursday	29.01	1	Meeting with main supervisor
Thursday	29.01	5	Group learning the Vivado simulation tool
Thursday	29.01	4	Home work: Vivado simulation tool and ARM theory
Friday	30.01	6	Home work: Vivado simulation tool and ARM theory
Wednesday	04.02	5	Group working on the pre-project
Wednesday	04.02	5	Home work: working on the pre-project
Thursday	05.02	1	Meeting with main supervisor about the supervision agreement
Thursday	05.02	5	Group working on the pre-project
Thursday	05.02	5	Home work: working on the pre-project

Friday	06.02	3	Group working on the pre-project
Friday	06.02	5	Home work: working on the pre-project
Tuesday	10.02	8	Home work: Vivado simulation tool and ARM theory
Wednesday	11.02	1	Meeting with client
Wednesday	11.02	5	Working with the group
Wednesday	11.02	5	Home work: Vivado, ARM theory
Thursday	12.02	1	Meeting with main supervisor
Thursday	12.02	5	Working with the group
Thursday	12.02	3	Home work: Vivado and ARM theory
Friday	13.02	6	Home work: Vivado and ARM theory
Tuesday	17.02	7	Home work: Vivado and ARM theory
Wednesday	18.02	5	Working with the group
Wednesday	18.02	5	Home work: Vivado and ARM theory
Thursday	19.02	1	Meeting with main supervisor
Thursday	19.02	5	Working with the group
Thursday	19.02	5	Home work: Vivado and ARM theory
Wednesday	25.02	3	Working with the group
Wednesday	25.02	6	Home work: Vivado simulation tool
Thursday	26.02	5	Working with the group
Thursday	26.02	4	Home work: Vivado simulation
Friday	27.02	6	Home work: Vivado and ARM theory
Wednesday	04.03	4	Working with the group
Wednesday	04.03	4	Home work: Vivado simulation and testing
Thursday	05.03	1	Meeting with client
Thursday	05.03	5	Working with the group
Thursday	05.03	6	Home work: simulation and testing
Wednesday	11.03	4	Working with the group
Wednesday	11.03	7	Home work: simulation and testing
Thursday	12.03	4	Group meeting and report writing
Thursday	12.03	5	Home work: report writing and continued testing
Friday	13.03	8	Home work: report writing and testing
Wednesday	18.03	4	Group meeting and discussion about further work
Wednesday	18.03	7	Home work: testing and report writing
Thursday	19.03	4	Working with the group
Thursday	19.03	6	Home work: writing presentation
Wednesday	23.03	2	Group work on writing presentation
Wednesday	23.03	7	Home work: writing presentation
Thursday	24.03	3	Group work on writing presentation
Thursday	24.03	5	Home work: working on presentation

Wednesday	25.03	3	Group work: practicing the presentation
Wednesday	25.03	0.5	Group performs midterm presentation
Saturday	28.03	8	Home work: report writing and Vivado testing
Monday	30.03	10	Home work: report and Vivado testing
Tuesday	31.03	5	Home work: report, reading about Vitis tool
Thursday	02.04	8	Home work: report, Vivado and reading about Vitis tool
Friday	03.04	6	Home work: report, Vivado and reading about Vitis
Sunday	05.04	8	Home work: report, Vivado and reading about Vitis
Monday	06.04	5	Home work: report, Vivado and reading about Vitis tool
Tuesday	07.04	6	Home work: report, Vivado and reading about Vitis
Wednesday	08.04	3	Group work
Wednesday	08.04	8	Home work: report
Thursday	09.04	2	Group work
Thursday	09.04	0.5	Meeting with supervisor
Thursday	09.04	8	Home work: Visual Studio program and CSAL
Saturday	11.04	8	Home work: Visual Studio program and CSAL
Sunday	12.04	8	Home work: Visual Studio, Vivado and OpenCSD
Tuesday	14.04	1.5	Practicing presentation in English
Tuesday	14.04	0.5	Presentation of our work with Volker and Quentin in France
Tuesday	14.04	7	Home work: report writing, Vivado and OpenCSD
Wednesday	15.04	3	Group work
Wednesday	15.04	6	Home work: report writing and design verification
Thursday	16.04	3	Group work
Thursday	16.04	6	Home work: design verification
Saturday	18.04	8	Home work: design verification, Vivado
Monday	20.04	8	Home work: Vivado, reading about Vitis and C code
Tuesday	21.04	5	Home work: Vivado, reading about Vitis and C code
Wednesday	22.04	3	Group work
Wednesday	22.04	5	Home work: EXPO poster
Thursday	23.04	6	Home work: EXPO poster
Saturday	25.04	8	Home work: VHDL, Vivado and reading about Linux tools
Sunday	26.04	10	Home work: VHDL, Vivado and reading about Linux tools
Tuesday	28.04	8	Home work: Vivado and reading about Linux tools
Wednesday	29.04	3	Group work
Wednesday	29.04	7	Home work: EXPO poster
Thursday	30.04	8	Home work: report writing, Vivado and reading about Linux
Friday	01.05	10	Home work: report writing, Vivado and reading about Vitis
Sunday	03.05	8	Home work: writing reflection note
Tuesday	05.05	6	Home work: Vivado and reading about Vitis and Linux

Wednesday	06.05	3	Group work
Wednesday	06.05	5	Home work: writing reflection note
Thursday	07.05	1	Meeting with main supervisor Endre
Thursday	07.05	4	Home work: EXPO poster
Thursday	07.05	8	Home work: report writing
Friday	08.05	7	Home work: report writing
Sunday	10.05	4	Home work: report writing
Tuesday	12.05	7	Home work: report writing
Wednesday	13.05	8	Home work: report writing
Monday	18.05	2	Group meeting
Tuesday	19.05	4	Home work: report writing
sum		545 hours	